IntentSpider - The Fluid Language Web for Textual Tension and Prediction

System Design, Gating System, Valence, Experiments and Validation

Anonymous Author

[email redacted]

Completed on July 21, 2026

IntentSpider Webnet is dedicated to the memory of NASA's Apollo 11, and to the American heroes who, on July 20th, 1969, became the first conscious beings from the Planet Earth to set foot on another world.

Abstract

IntentSpider is an on device, privacy first predictive text engine based on the C++ standard template library, built as a personal intent graph for individual humans. The prediction and graph update processes operates inside the device, making the engine offline accessible. The web research preview/playground can separately use a Global Person Mode and a database connected state, but this is separate from the IntentSpider prediction engine. Every edge consist of a coefficient and a transmission fidelity value which branches the logic into whether that edge is a TF type or a capture type, according to the probability of the signal traveling towards the human's next selection. These roles are reversible. IntentSpider uses a personalized PageRank algorithm through the local push process. And it is initially seeded from several recent words using fan dispersion algorithm rather than the same starting values. These function typed connections and the fan dispersion seed are both motivated by the dynamics of the spiderweb.

IntentSpider core also has one structural problem denoted as the blindness to the valency. Valence Gated Signed Diffusion adds a valence signal from typing interval patterns and correction behavior. The Necessity Gating algorithm determines and executes the logic controlling whether an already ranked next word should be displayed or hidden. Hexarousal, adding another signal from the amount by which the current typing rate differs from that human's general, ordinary typing rate average. In addition to this, IntentSpider includes shared reinforcement, an increasing diffusion radius, sudden suppression algorithm, settling, typing substate search, and combined behavioral values, motivated by the Orb Spider, Wandering Spider, and the struggling isopod prey. As a separate component, the 2D vector converts sentence diffusion outputs into Geometric Paths.

Across 500 evaluation events, IntentSpider reached 14.4% first rank accuracy and 27.8% Top 3 accuracy. On all shared event sets, IntentSpider produced greater first rank (measured) and Top 3 accuracy, higher, measured accuracy (in this evaluation) than Google Gboard, Samsung Keyboard, Microsoft SwiftKey, and OpenBoard. Against Gboard on 187 shared events, IntentSpider reached 17.65% against 7.49% first rank accuracy, and 33.16% against 14.44% Top 3 accuracy. Prediction produced a median time of 2.4 milliseconds and a 3.66 ms with a 95th percentile. As the paper explains further, the greatest changes to those results appeared when the recent word seed was reduced to approx. 1, and when the Necessity Gating Algorithm was removed. This depicts the uncertainty in modern next word prediction, potentially altering, simplifying or influencing human communication patterns. IntentSpider Webnet is

one proposed approach to this unresolved issue.

Keywords: ML, PageRank, Graph Diffusion, Biometrics, Dynamic Programming, Graph Theory, Temporal Graphs, Privacy, Predictive Text, Gating Systems, Spectral Graph Theory, Human-Computer Interaction, Word Prediction

---

Sections, Subsections and Contents

1. Introduction

Software keyboards such as Google Gboard, Microsoft SwiftKey, Apple QuickType, etc., predict a next word from the text which came before it. Their complete internal architectures are not the same, and many of the production parts are private. In papers, one important sequential background for this problem is the classic n gram language model. Katz back off uses a shorter context whenever the longer one is missing, and Kneser Ney smoothing redistributes probability across contexts [14, 17]. And also, current keyboard systems can use recurrent, neural, federated, personalized or other processes. IntentSpider takes a separate direction in here. It stores the earlier information as a changing personal web of word connections, and then the graph processes described below read that web.

Production systems can also learn a shared model while keeping the raw keyboard text inside the device. In Google's published federated next word research, for an example, the raw keyboard input stays inside the device [10]. Only the combined model updates from many devices are transmitted to the server. IntentSpider uses the one human's changing graph itself as the prediction state instead. The core graph update and diffusion can operate locally, without needing a remote inference request for that prediction. And also, the web playground used in Section 10 adds a separate process for saving and loading the serialized state around that local engine. Which means that the prediction engine and the saving process are separate parts in this implementation.

IntentSpider is an on device, personalized alternative to those existing systems. It is a system that represents the word units typed by one human as nodes of a personal intent graph. In the current tokenizer, those units are individual alphanumeric words and apostrophe words. Several adjacent words are not combined into one graph node in the current system. Each edge is scored by the accuracy of that connection, meaning how reliably it helps reach the next word the human actually enters. This score is a number that can rise and fall, rather than being fixed by the age of that edge. The web playground can save the graph through its separate state saving process, while the C++ prediction and graph update itself does not require remote inference. Then, IntentSpider sends the fan dispersion based seed outward through only the edges which currently have a transmission type status, using a personalized PageRank type graph diffusion [16, 13, 1]. For that reason, a word combination which has not appeared in that exact order can still receive a next word through other reliable connections already present in the personal graph. This process requires an already existing graph region to reach. And also, this component replaced an IntentSpider legacy version which instead handled two separate connector types operating with a difference in timescales. [19] [11, 21, 2, 22]

The name IntentSpider comes from this same web structure. The main object in here is the changing web of word connections. The function typed edges act similar to threads with separate purposes. A transmission type edge can carry the prediction signal further, while a capture type edge remains as a direct local association. The diffusion process reads this web by sending the signal through the connections which have earned the relevant role. An earlier observation of a disturbance traveling through a layered spider web contributed to the idea of reading a changing network through its transmitted signal. And also, the observation that already caught objects can change a later disturbance contributed to the residue and support choosing process. The separate silk purposes contributed to the present function typed edges. The project design history records this origin (refer [38]). These are the biological ideas which contributed to the engineering process. The later attacking and struggle analogies in Section 6.9 are separate processes built around this same IntentSpider system.

Also, the IntentSpider prediction process does not use a pretrained language model, trans-

former, neural embedding model, part of speech tagger, parser, semantic word list or another externally supplied linguistic representation. Which means that its next word process is made from the word identities, the changing graph, the recent input, and the internal graph and typing signals described in the following sections. Due to that reason, Section 10 compares the suggestions produced by the complete keyboard systems as those systems were used in the experiment.

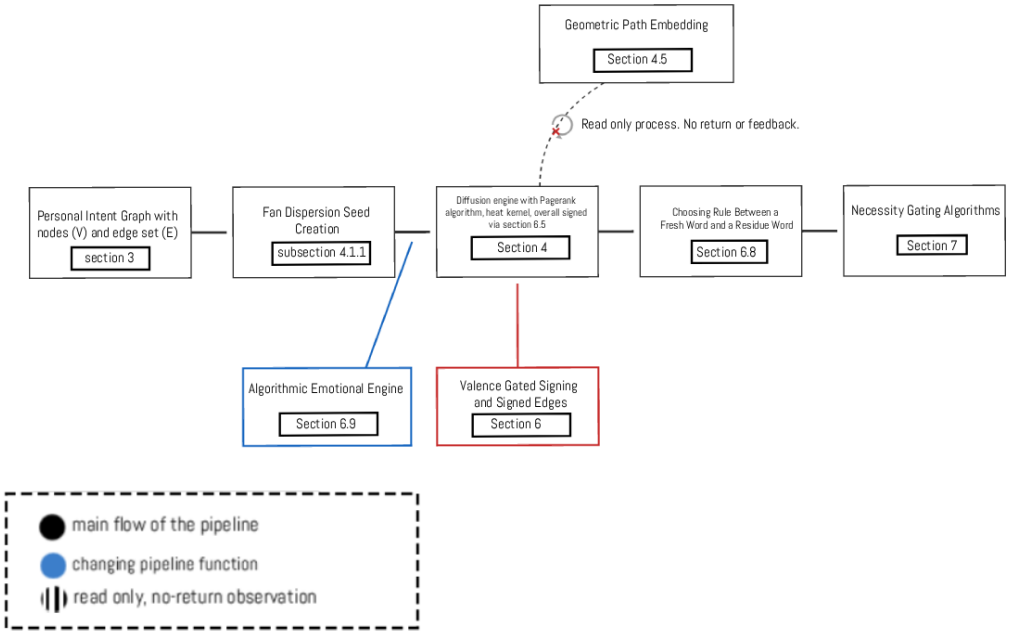

Figure 1. An Overview of the IntentSpider processing architecture

1.1 Contributions

1. It gives an accurate description of a diffusion architecture with assigned roles to them, In this architecture, edge's role in sending signals is assigned by the reliability of that edge so far, rather than using timers or clock function., And also, The novelty of the IntentSpider is compared with many other research and related work including the two timescale and elastic coefficient consolidation work (or EWC), which proposed in IntentSpider legacy version.
2. IntentSpider solves a structural problem in the IntentSpider legacy base architecture. This problem can be named as the blindness to the valency. And also, in this problem, neither the raw coefficient value update nor the reliability value update can separate a next word selection that confirms what the user meant from the next word that makes the text exchange with the other party final.
3. IntentSpider also gives a separate and independent process for filtering, which is "necessity gating" process for deciding whether a next word should be displayed or chosen. And also, this decision is separate of next word that the processes of Section 3 and Section 4 would rank first. And also, IntentSpider generates this necessity gating mechanism through a creative use of the SCAMPER method (Substitute, Combine, Adapt, Modify, Put as a one, Eliminate and Reverse, not Rearrange.), rather than from a new failure mode found in the system.

4. The current IntentSpider implementation is evaluated in Section 10 using fixed next word evaluation data, four existing keyboard systems, processing measurements and controlled changes to the IntentSpider processes. In the complete 500 event result, IntentSpider reaches 14.40% first rank accuracy and 27.80% Top 3 accuracy. And also, in the event sets available for both systems, IntentSpider has the greater measured first rank and Top 3 accuracy against Gboard, Samsung Keyboard, Microsoft SwiftKey and OpenBoard. The same section measures the necessity gating process and several other IntentSpider processes separately as well.
5. There is a second system that operates separately like as the necessity gating architecture. Named as the 'affective axis', this can also be named as "hexarousal" alongside the valence signal.
   In addition to that, Section 6.9 defines the processes created from the attacking and struggle analogy of the spider and the isopod prey. The first is reinforcement that compounds across recently selected word connections. Another is a growing diffusion radius. Another is the sudden negative selection response, and another is the settling process after that temporary state. The same section also finds the distinct per user typing sub states that the human's current typing shows. And Equation (34) gives a combination of the measured typing features. These are separate timing based processes around the graph.
6. Also, this gives the same graph a 2D spectral embedding (refer to section 4.5). In the native developer process, the diffusion outputs from the word prefixes of a sentence create a path through a plane. This process is read only. Which means that it gives a debugging and inspection view from diffusion outputs IntentSpider already produces, rather than changing the next word process itself. This includes the distance of a node from the center and the reconvergence of two paths around a shared word.

Figure 2. An orb weaver web weaving spider on a spiderweb surrounded by dead/trapped prey and with multiple other spiders.

The above figure represents one of the main natural design principles used in the IntentSpider system. It includes the assumption that vibrations from newly struggling prey travel through the web to the poorly sighted web weaving spider, while being dynamically influenced by other ”weights” or ”residue,” described later in the paper, as part of the mapping process.

2. Related Work to IntentSpider

2.1 The Sequential Processing Background of Current Keyboards

In text prediction systems, often, there are two types of techniques common in papers. The first one is classic n gram language modeling with smoothing. Katz back off model uses a shorter context whenever the longer one is missing and/or not accessible. [14] Kneser Ney smoothing [17] directs the probability to contexts that are more useful, not only the ones which are frequent in use. They are foundational to these systems because they have a small memory and cost, and they give one of the established sequential backgrounds for current next word prediction systems.

Andrew Hard, affiliated with Google Gboard, trained a recurrent next word model for a mobile keyboard using the Federated Averaging algorithm. In that system, the raw keyboard input remains inside the device. [10] The model updates from many devices are combined before they are transmitted to the server. This research partially motivated us to keep the IntentSpider prediction process local to the human's own device. A federated keyboard can still include local personalization as well. IntentSpider takes a different route in here. It uses the one human's graph itself as the prediction state, and the core graph update and diffusion does not require remote inference. The web playground used in Section 10 adds a separate process for saving and loading that state.

2.2 Usage of two memory systems or Dual Timescale Systems and the process of learning

The theory of two interconnected memory systems running at different speeds is an old concept. The concept did not start from a ML background. In cognitive neuroscience, McClelland, McNaughton and O'Reilly [19] describe Complementary Learning Systems or CLS as complementary learning between a hippocampal system which can acquire new episodic information rapidly and a neocortical system which learns more gradually across experience. This difference matters in here because an earlier IntentSpider design also separated a fast changing state and a slow changing state. In here, the biological memory systems and the software implementation remain two separate systems. CLS is used as the related background showing that multiple learning timescales are already an established idea. The current IntentSpider architecture no longer uses that split as its main memory structure, but the theory remains important as the background of that earlier design and the comparison systems used later in the paper.

Professor Geoffrey Hinton and Dr. David Plaut [11] described a connectionist system where each connection carries a slowly changing weight and a fast changing weight which decays back toward zero. Their experiments showed that the fast weight can temporarily counter interference which had accumulated in the slower weight. Which means that, keeping one fast coefficient and one slow coefficient is already an established construction and cannot by itself be the novelty of IntentSpider. In Section 3.2, the current system replaces that earlier two channel split with a single edge set. The edge's function is decided by its ongoing transmission fidelity value instead.

And also, recent continual learning systems continue to use this fast and slow pattern in different forms. DualNet [21] separates fast and slow learning processes. Complementary Learning Systems based Experience Replay or CLS ER [2] combines episodic replay with short term and long term semantic memories which update at different rates. Information Theoretic Dual Memory System or ITDMS [22] keeps a fast memory buffer for newly acquired samples and a slow memory buffer selected to preserve diverse and informative samples. These systems are not the same system, and they are not the same as IntentSpider. But all of them give examples of the same fast and slow memory pattern already being used in the literature. Which means that, a novelty claim based only on that specific fast and slow split

would be weaker. The current IntentSpider architecture does not use that split as its novelty claim.

2.3 Protecting System's already learned information

Elastic Weight Consolidation or EWC supplied the parameter protection idea used by an earlier version of IntentSpider [15]. For each parameter, EWC estimates how much that parameter mattered for the earlier tasks using a Fisher information approximation. After that, it adds a penalty against changing the parameters which carried more of that earlier importance. Which means that the earlier learned information can be protected by making some later parameter changes more costly. An earlier IntentSpider design used this general idea when an edge moved into a more stable state.

But the current IntentSpider architecture does not use EWC as a system mechanism. Section 3.2 gives the edge its forwarding role from transmission fidelity instead. That transmission fidelity value can move above or below its margin as later prediction outcomes arrive, and therefore the functional role can move in both directions. In this comparison, EWC remains the parameter protection method and transmission fidelity is the reversible functional edge role. EWC remains in this paper because it belongs to the earlier IntentSpider design, and also because it is used as the source for baseline B3 in Section 10.2.

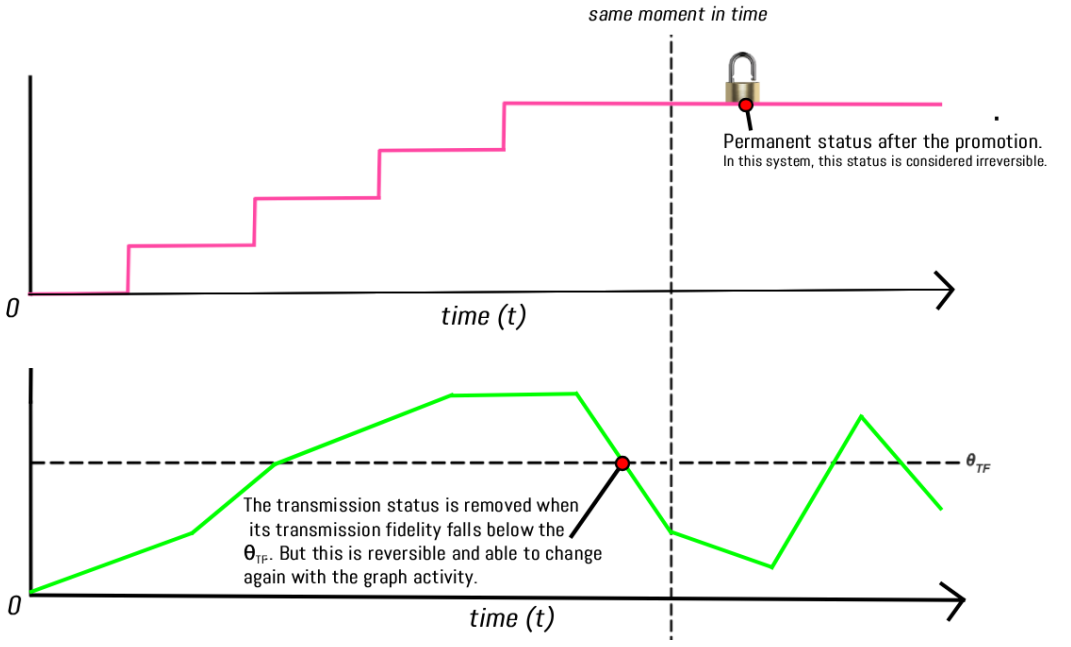

Figure 3. Flexible TF state changes. These changes are two way. Reverse possible.

2.4 Graph Diffusion System and Usage of a Personalized PageRank

Personalized PageRank builds on the original PageRank algorithm developed by Larry Page and Sergey Brin [20], more specifically its random walk model. In here, instead of restarting the process in the same way across the complete graph, the personalized form can restart toward one specific node or a seed distribution [13]. Klicpera, Bojchevski, and Günnemann [16] used personalized PageRank propagation in PPNP and APPNP to carry neural predictions over a graph. IntentSpider uses the same diffusion family for a different place in the

process. Its seed comes directly from the recent word nodes, and the ranked output is read from the diffusion itself instead of from a neural classifier which came before it.

In addition to that, there is another related option used in Section 4 which is called the heat kernel formulation. Professor Fan Chung [4] analyzed the relationship between the heat kernel diffusion and PageRank diffusion. These two methods are related, but they do not use the exact same diffusion kernel or the same weighting series. Due to that reason, Section 4 keeps both of them as related graph diffusion constructions instead of treating them as the same vector after only a different amount of elapsed time.

2.5 Background for Using Local Computation for Practical Applications

IntentSpider's use of a PageRank variation random walk model, and computing an exact user based PageRank vector, means solving a system of linear equations across the entire graph here. This requires a massive amount of processing if it is repeated continuously. In the WebAssembly process used in Section 10, the raw keypresses update the typing timing values. The graph diffusion and next word prediction are executed when the current word is committed. Which means that solving the complete graph again for every one of those prediction events would still require a much larger processing amount.

Reid Andersen et al. solved a version of this problem previously in 2006 (refer [1]). Their local push algorithm computes an approximate personalized PageRank vector instead, by pushing mass outward from an initial node. Here, in this process, 'mass' is only added to nodes which still carry more than a small, fixed amount of unused value. And also, the entire process finalizes in time which is proportional to $\frac{1}{\alpha\epsilon}$ and this time is not dependent on the number of nodes on the graph. This is the reason that Section 9 studies the local computation of the system. Without this local approximation, one prediction can require processing across the complete graph created from the human's earlier words. Section 10 gives the measured processing time from the browser environment used for the experiment.

2.6 Strengthening and Suppressing Edge Connections in IntentSpider (Graph Edges)

Ant Colony Optimization, or ACO, takes a massive space in terms of initial cointributions to the IntentSpider development and research, which means that the edge strengthening and edge decay dynamics used for the raw coefficient system in Section 3.2, this methodology came directly from the Ant Colony Optimization (refer [5]). In this method, pheromone trails on a graph get reinforced by traffic (the analogy of ants moving through a field of clay), and these trails of the ants' foot fade according to the time passed so far when they are not used.

And also, after a pattern completes, IntentSpider keeps the exhaustion value from Section 3.2 as a short record of that recent selection. The Tabu Search research (refer citation [9]) motivated the idea of discouraging an immediately repeated solution. In the current IntentSpider process, the exhaustion value does not literally block an edge and it does not stop diffusion through that edge. Section 7 reads this value as one input which can reduce the Necessity Value before a ranked next word is displayed. (refer to section 3.2 and section 7)

2.7 Signed Graph Connections, and measuring The System's Emotional Tone Signal (or Valence)

In Section 6, it needs a programmable object for a type of graph. In this specific graph, some edges should actively push back against a spreading signal. And also, this push back is different from the general type of fail. Which means that it is different from the failure to reinforce that signal.

In the fundamental graph theory in math, this is not novel. Professor Dorwin Cartwright and Professor Frank Harary [3] extended Heider's structural balance theory into graphs with positive and negative relations. In addition to that, Jerome Kunegis and others [18] developed spectral methods for signed networks, including a signed Laplacian used for clustering, prediction and visualization. Due to that reason, these works give the signed graph background for the negative relation used in Section 6. The IntentSpider signed local push process itself is described later in that section.

An earlier IntentSpider version used VADER, referred in the research as the Valence Aware Dictionary and sEntiment Reasoner, as a light text valence source [12]. VADER is a rule based sentiment model and not a neural network based model. The current IntentSpider system no longer uses that semantic word list for its runtime signed signal.

Unlike the IntentSpider legacy version, Section 6.2 replaces VADER with a separate signal. This new signal is based on the specific typing interval pattern of the human, and it does not require a previously created list of emotional words. This signal is motivated by the spider neuroethology study, which researches how a spider's nervous system produces responses (refer [26]). The VADER paper remains cited because its valence process was one of the sources used while creating the earlier IntentSpider theory. VADER reads emotional meaning from written text. The current IntentSpider process instead uses the typing interval pattern and the correction rate, keeping the semantic meaning of the word outside that calculation.

2.8 The Reading of Affect and Emotional Tone in Text Communication Between Humans

Aligned with this, there are two already existing research papers which support the affective framework being discussed in Section 6.9. These citations also give the background for why the later calibration is kept dependent on the communication setting.

The first step is representing affect as a small space with independent axes rather than as one single number. Professor James Russell [28] proposed representing affect using two dimensions, valence from pleasant to unpleasant and arousal from activated to deactivated, arranged into a circular model. This research belongs to 1980. (refer section 6.9.1.)

In addition to that, Professor Joseph Walther introduced the Hyperpersonal Model [29], which explains how online text communication can differ from face to face interaction through properties such as selective self presentation, receiver interpretation, channel constraints and feedback. This matters to IntentSpider in one specific way. A typing behavior calibration created in one interaction setting should not automatically be carried into another setting as the same value. Which means that a calibration made for instant text messaging can require a different setting from one used for email, note taking or search. Walther's model gives no one numerical value to use for those different interfaces. Due to that reason, Section 6.9.8 keeps that channel coefficient as an IntentSpider setting instead of taking a number from Walther's model. And also, the six other mechanisms in Section 6.9, compounding reinforcement, growing radius, shock retreat, settling, sub state clustering and the nonlinear interaction coefficient, remain IntentSpider mechanisms rather than values taken from Russell [28] or Walther [29].

2.9 Mapping and Placing Graph Connections Into a Smaller Space (Related to Spectral Graph Theory)

In Section 4 of the IntentSpider paper, Section 4.5 uses an already existing spectral graph idea. Which means that the structure of a graph can be represented in a smaller dimensional form from the eigenvectors of its Laplacian. Professor Miroslav Fiedler [30] showed that the second smallest Laplacian eigenvalue gives a one number measure connected with how strongly the graph remains connected, which is known as the algebraic connectivity. The

eigenvector connected with that value, which is commonly called the Fiedler vector, can give a direction for separating the graph. Which means that the eigenvalue and the eigenvector are used for two different parts of this interpretation.

Professor Mikhail Belkin and Professor Partha Niyogi [31] developed Laplacian Eigenmaps. In that method, data are represented using the eigenvectors connected with the smallest non trivial eigenvalues of a graph Laplacian or its generalized eigenproblem. This produces a low dimensional embedding while keeping the local graph structure in place. It is related to, but distinct from, PCA and classical multidimensional scaling.

Section 4.5 applies this established technique to the changing personal IntentSpider graph. In this paper, it is used as a debugging and inspection view. From this, it reads two geometric relationships. The first is the connection spread of a word against its distance from the center. The second is the reconvergence of two paths around a shared word. These are measured from diffusion outputs which IntentSpider already produces.

2.10 Novelty of the IntentSpider Webnet

After reading from section 2.2 to 2.9, one of the conclusions that we will be able to take is that text prediction, and more specifically on device text prediction, is an established area of research in human computer interaction. In addition to that, the theory of two timescale memory architecture is an already existing area of research with several published architectures. Those architectures use Complementary Learning Systems ideas which also influenced the earlier two timescale IntentSpider legacy design. The current function typed IntentSpider architecture replaces that earlier two channel split. In addition to that, protecting learned parameters, diffusion based classification, local computation inside the device, reinforcement and dynamics of inhibition, signed graph methods, keypress dynamics biometrics and spectral graph embedding are also previously researched concepts into a some extent.

And also, several biological findings related to Orb weavers and the research related to them support the natural ideas used in this design. Orb weaver silk comes in separate types, and each type of silk has a different function. Tiger wandering spider's (scientific name: Cupiennius salei) responses can change with vibration frequency, including approach and withdrawal behavior. Professor Russell's circumplex model also gives the valence and arousal background used later in the paper.

The figure below demonstrates vibration receptors and the leg like structures around that area connected with the vibration responses.

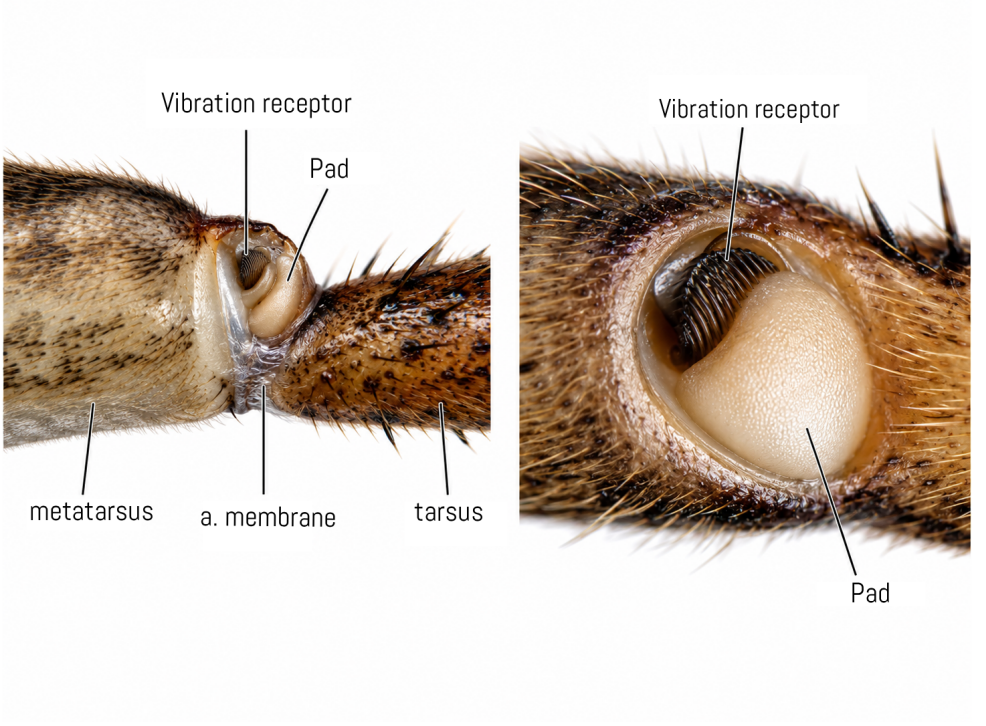

Figure 4. Process - The vibration receptor in the metatarsus part and tarsus connect with the pad. This receptor provides the sensory structure for sensing. This is also connected with a diverse array of return responses.

We have not found a system that types graph edges by an ongoing, freely reversible measure of whether they actually predict what a user goes on to select. This is different from typing edges by elapsed time, usage frequency, or a one way consolidation clock. And also, we have not found a system that reads that typed graph out through diffusion, seeded by a combination of several recent words coefficient based by how scattered their connections are, rather than by the single most recent word. Also, we have not found one that adds a signed, valence gated extension whose valence signal comes from behavioral timing rather than word content.

Those three processes are the main architecture claim made by IntentSpider Webnet in this paper. The first is the freely reversible function typing of the word connections. The second is the fan dispersion weighted seed made from several recent words. The third is the typing interval pattern based signed extension. Section 10 evaluates the current predictor and also changes several IntentSpider processes separately while keeping the evaluation data the same.

3. A Novel, Personal Intent Graph with Function Typed Edges

3.1 Runtime and System Overview

IntentSpider system is operating overall as a graph for a single human, and the prediction and graph update processes can operate inside one device. For this prediction engine, a remote inference server is not required. And also, there is no combining of several humans' graphs into one larger Webnet. This is a self imposed design constraint and it is the local privacy oriented architecture discussed again in Section 8. Which means that the graph described below is built from that specific human's own typing history.

The web playground used in Section 10 can separately load and save a serialized IntentSpider state through a server. Due to that reason, the prediction engine and the state saving process are two separate parts. The stored graph itself is not presented as a cryptographic or differential privacy process.

3.2 Nodes, The impact on function based on the specific thread (or silk) type

Orb weaver spider webs are built from physically different silk types, and those silk types have different mechanical functions [25]. The radial and frame parts support the web structure and transmit vibration through the web. The capture spiral has a different purpose and is specialized for keeping the prey. Which means that the thread's purpose or role is connected with what that thread does, rather than only with how long that thread has existed. IntentSpider uses this point as the engineering analogy, while the actual edge role is calculated from transmission fidelity. It replaces the earlier time based channel split with a function based one. An edge's role in the graph is determined by whether it reliably carries signal to a useful place, not by how much accumulated support value it has built up over time.

Let V be the set of tokenized word units that this specific human has typed. In the current IntentSpider process, these are lowercased alphanumeric words which can also include an apostrophe. The current tokenizer keeps those words as individual nodes. Section 4.1.1 separately discusses the possibility of combining several words into a one node. This set grows as new words appear. Rather than two coefficient functions across two separate edge sets, IntentSpider now keeps a single edge set "E" on "V". Every edge, referred to as u and v, in this set carries the values described below, and each of those values updates separately.

Raw coefficient. A selection event in here is the next word v committed after context u. This can happen when the human types that word or accepts a suggestion. At that event, the raw coefficient is reinforced, or strengthened. In other cases, it starts to decay in exponential value according to the time since it was last used. This works in the same general way as the pheromone trails of the ants inside the Ant Colony Optimization algorithm, or ACO algorithm (refer to [5]).

$$w(u, v, t + \Delta t) = w(u, v, t) \cdot e^{-\Delta t/\tau_v} + \eta \cdot \mathbb{1}[\text{selection event at } t + \Delta t] \tag{1}$$

In this context 'τv' represents a short sized decaying constant, referred as the Tau sub V constant, which is programmed to be active between several hours and several days. This specific speed decides how fast the strength of a word connection lowers when a human does not choose it for a specific period of time. Which means that the influence of an unused association can reduce over time. A connection is not automatically deleted only because this raw coefficient became small. This specific symbol 'η' refers to the learning rate used by this process. This learning rate decides the amount by which the coefficient of a specific connection increases each single time when a human selects a word. Which means, this controls the speed with which the engine adapts to new typing patterns and the human's personal preferences.

In addition to that, this above Equation (1) updates one single edge for each committed next word event. Below in Section 6.9.2, this paper introduces an additional word group strengthening term. This term layers on top of the existing update rather than replacing it. In the current IntentSpider process, it applies when more than one recently selected word connection remains inside the short active period. A word which only received diffusion mass is not added into that active set. This additional reinforcement process uses its own initial reinforcement value. And in here, there is a separate symbol for this value in order to avoid confusing it with η in the other calculations.

To manage the repeated recent selection, the system keeps a Tabu Search motivated value named exhaustion. After a selection event, this value begins near 1 and starts to decay toward 0. The Tabu Search research discussed in Section 2.6 motivated this short record of a recently selected solution. In the current IntentSpider process, this exhaustion value does not stop diffusion through that edge. Section 7 uses it later as one input to the separate Necessity Value which decides whether the ranked result should be displayed.

$$\text{exhaustion}(u, v, t) = \exp\left(-\frac{t - t_{\text{last}}(u, v)}{\tau_x}\right) \tag{14}$$

In this Specific case, '$t_{\text{last}}(u, v)$' represents the time of the most recent selection event on edge (u,v), and τx represents the short decay period of the exhaustion value, which Section 2.6 explains in more detail (refer [9]). This τx symbol is separate from the "τv" value which decides the raw coefficient decay in Equation (1). And also, it is separate from τ0 introduced in Section 6.8, which controls the recent support decay used by the choosing process. exhaustion(u, v, t) is a separate accounting signal by itself. (As a note, this specific signal remains closer to the 1 that specific moment after the selection. and as the time spends, that specific value starts to decay down to 0). It is not subtracted from w(u, v) in Equation (1). And the Section 7 uses this signal downstream. In section 7, there is a signal in order to decide whether a candidate should be shown but it is not used in order to change any already stored edge coefficient.

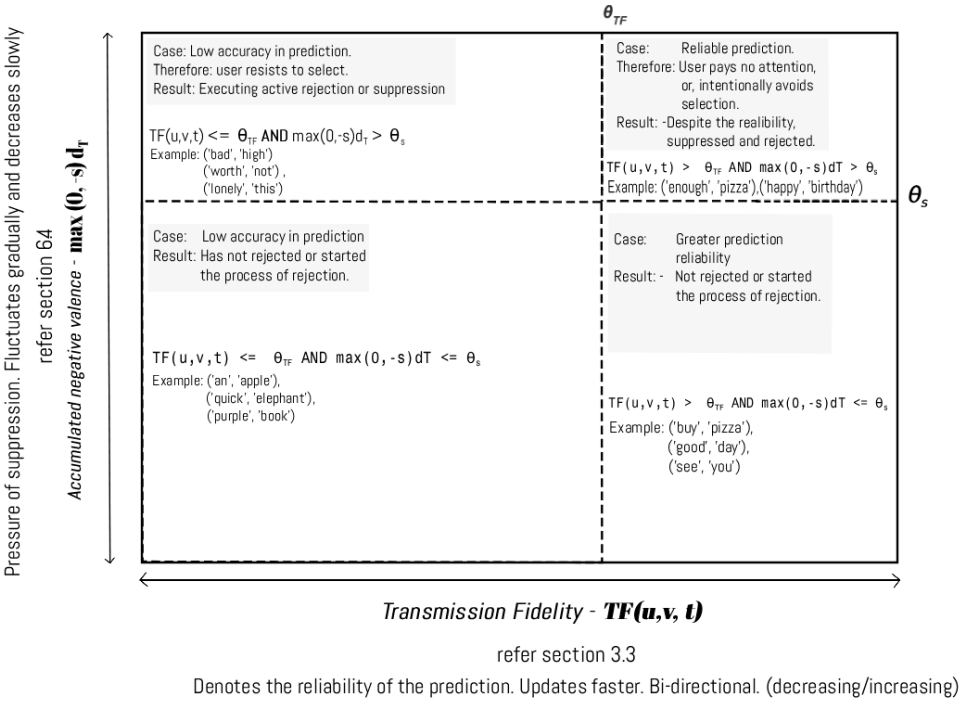

Figure 5. In the figure above, the TF measures the reliability of the prediction.

This section introduces and defines transmission fidelity. in this theory, every edge object includes a transmission fidelity value, which is written here short as "TF", and this value is built with u, v and t and remains between 0 and 1. The prediction made before the current word is resolved when the next committed word arrives. At that moment, an edge is included in the TF outcome update when it carried at least the non trivial diffusion mass amount used by the engine. If the destination of that edge is the word the human entered, its outcome is 1. If its destination was one of the ranked words but was not the entered word, its outcome is 0. Which means that the TF value changes from the result of a prediction event and not only because time passed.

$$TF(u, v, t + \Delta t) = TF(u, v, t) + \eta_{TF} \cdot (\text{outcome}(u, v, t) - TF(u, v, t)) \tag{16}$$

3.3 Edge Role Assignment Process and Theories

$$(u, v) \text{ is } \begin{cases} \text{transmission type} & \text{if } TF(u, v, t) > \theta_{TF} \\ \text{capture type} & \text{otherwise} \end{cases}$$

Equation explanation - In here, the transmission type edge is included in the diffusion matrix $W_{\text{transmission}}$ and enables the diffusion process. The capture type edge is excluded from that forwarding matrix and remains as a direct and non forwarding association. In addition to that, if the diffusion ranking returns fewer than three words, the current IntentSpider process can fill the remaining positions from unsuppressed direct outgoing edges of the most

recent word. Those capture type connections remain direct associations only, and they do not forward the signal again from that next word.

The EWC's importance value which is referred as 'F' takes in u value and v value combined and afterwards, this value is only increasing the equation 2 (Eq. (2)) whch was in the IntentSpider legacy version. This specific process resulted in the edge that was complex, limited and reduced the ability of the capacity of the edges to change. But the TF val, or transmission fidelity value, and then processing value "v", "u" and "t" as a single array of data can move in either of those directions as the human's typing patterns change. And also, an edge that fails to provide the value to the system results in a reduction from its transmission status. Which means that none of those role assignments are permanent in this system. After that, the legacy IntentSpider version was not capable of unlearning the already learned but falsely stable and also outdated associations and information due to a specific lack of a process for automatic and more natural cleaning. This is also discussed in section 2.3 and equation no. (2) and no. (3). To recorrect this issue, equation (16) and equation (17) invents a new process for unlearning.

This section is about the reliability of the above processes. In the current working values, a new edge begins with TF = 0.5 and $\theta_{TF} = 0.35$ is used as the transmission margin value. Which means that a newly created edge begins as a transmission type edge. Later prediction results can move this TF value downward or upward. If that value reduces below the margin, the edge becomes a capture type. And if the reliability increases again, it can return to the transmission type. In addition to that, the legacy IntentSpider version tracked the recent and the reliability as a combined object on a single promotion clock. This IntentSpider version separates them into separate quantities.

This section uses η_{TF} as the main free parameter for this process. This processing system does not require a fixed window "T" since "TF" calculates an ongoing exponential moving average instead of a specific windowed count. And also, compared to the legacy version of IntentSpider (means, before the revision), this system reduces the number of free parameters by one

3.4 An Overview of the Architecture in Figure 1

In here, a new input for example, such as a pizza order, accumulates a raw coefficient and after that begins obtaining a transmission fidelity value (alt. referred as the TF Score). An edge whose transmission fidelity passes θ_{TF} becomes a "transmission type" and added to the process diffusion (refer section 4). An edge that does not pass this specific margin value is a "capture type" and then remains available only as a direct, local, immediate and non forwarding association. This architecture is different from the previous architecture with two channels. And unlike in legacy version, in this architecture, movement between these two roles is continuous and reversible in both directions as $TF(u, v, t)$ value changes.

4. The Process of Diffusion

The diffusion step decides how a newly typed word, or several of those words, produces the ranked suggestions. In this process, transmission type edges ($W_{transmission}$, refer to Section 3.3) are used only for propagation. And also, capture type edges absorb new input, but those capture type edges are intentionally excluded from the long range diffusion. Meaning a transient, situational, or unreliable pattern cannot flood the suggestions in the same process a reliably predictive one can. As a note, this paragraph, and also the heat kernel and the personalized PageRank equations below, are unchanged from the earlier two channel version of IntentSpider Legacy version.

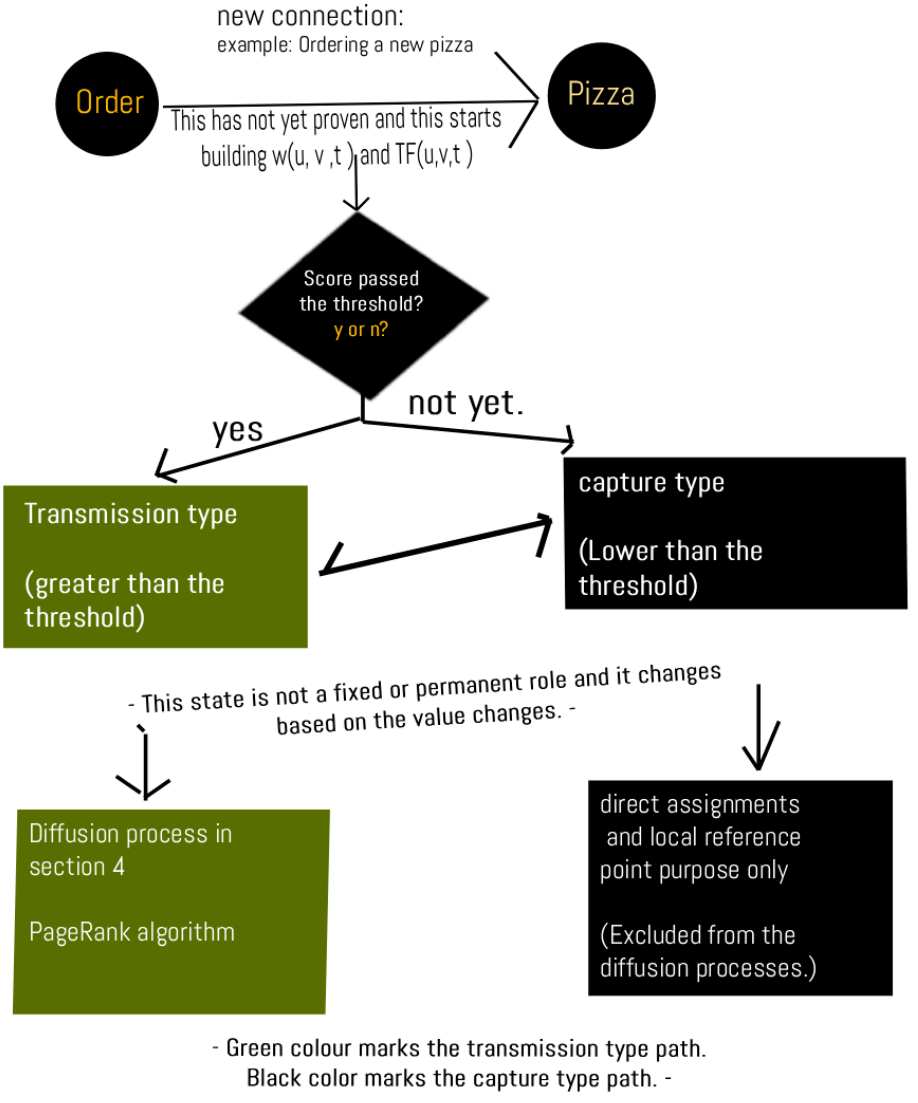

Figure 6. Above figure depicts the single, functional and function typed edge set and the main, two states of possibilities

4.1 Heat Kernel Formulation

Let L = D – $W_{transmission}$ be the graph Laplacian of the transmission type edge set, and in this case, D is its degree matrix. The heat kernel at the diffusion time t is

$$H_t = \exp(-tL), \qquad s(t) = H_t \cdot s(0) \tag{4}$$

In this scenario, s(0) is a seed vector, which is constructed as described in Section 4.1.1 right below it, rather than as a 0 or 1 value indicator on the single currently active word. Which means that, as "t" value increases, the signal is shared via using the transmission type edges. And also, nodes which are strongly connected and nearby receive relatively more signal than the ones in the further distance. Professor Chung (refer [4])noted the precise correspondence between this continuous time process and PageRank algorithm. And this specific correspondence is the reason which licenses treating the two formulations in this section as can be substitutable for one and the another.

4.1.1 Creating Seeds from the Node Dispersion Algorithm and Using Shannon Entropy in the Information Theory to Rank Previously Typed Words by the Human

A seed vector with a single 1 at the just typed word is considering every word as a equal and a similar origin or a starting point for the diffusion processes. Regardless of the specific state of the connection or th connectivity of that specific word, this process executes. Meaning that, this method does not consider and it actively does not include whether the usefulness of the specific signal that it carry.This also does not include whether the word connects to many other words that the combined and collective signal is not clear. In addition to that, in English, connecting words like 'the' or 'in,' and with other completing or filling sentences such as 'you know', 'of course', 'by the way' are soem specific examples of this problem. Which implies these objects (sentences or words can be considered in a high probability and less variance, as they can be used before or after many types of words.). Due to that reason, if one of these words is used by itself to begin this process, the signal reaches every part of the entire graph in equal amounts. And it is not moving to the human's underlying intent.

For node u, let π(u, ·) be the its relative outgoing coefficient distribution, limited to the $W_{transmission}$ edges. This limit is the specific point of integration with the previous subsection no. 3.2. Meaning that, in here, the system only measures the dispersion values for the edges which are already in the state "transmission type" Hence, a relevant node's dispersion value is showing the correlating shape of its reliable connectivity. And it does not showcase the raw type connectivity(without shape) .

If we define

$$\varphi(u) = \frac{H(\pi(u, \cdot))}{\log(\deg_{transmission}(u))} \in [0, 1] \tag{18}$$

This uses the "Shannon Entropy " which is a basic theory discussed in the Information Theory. And before this, we introduced Shannon entropy H(·). It is also for the ranking confidence signal in the section 4.4 below. And for a second role, it is now again reused. Means that, this does not evaluate the confidence values of the output distribution but rather, it is the dispersion of the specific input node's internal local connections profile. If a word's transmission edges spread evenly across many other words, φ(u) equals a value which is close to 1 approx. If a word's transmission edges focus on a few amounts of next words, φ(u) equals the value close to approx. 0.

The seed vector for the last "k" typed words u_t, u_{t-1}, …, u_{t-k+1} is then

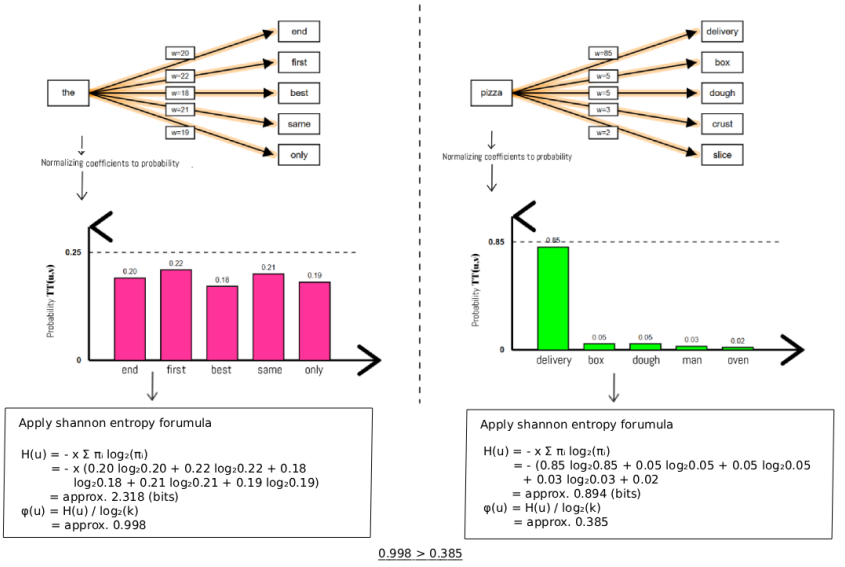

Figure 7. High and low dispersion node connections and their Shannon entropy values.

$$s(0) = \text{normalize}\left(\sum_{i}(1-\varphi(u_{t-i+1}))\cdot e^{-(i-1)/\tau_{seed}}\cdot e_{u_{t-i+1}}\right) \tag{19}$$

τ_{seed} does not share a symbol with any wall clock decay constant in any other place as well. Those are τ_v, τ_x, and τ_0. And the subsection 6.8.3 is similar for ρ and γ, for that similar reason. τ_{seed} is reducing the word's specific role or the part in the seed as that word's position in the typed sequence moves further back. But it does not reduce based on elapsed real time. A word with no transmission type edges has an undefined $\varphi(u)$ value. And this word does not need a special case in Eq. (19). Which means that, this kind of a word cannot send signal. Due to that reason, it does not matter what seed coefficient the equation gives to this word. And the equation naturally gives this word no effect.

Equation (18) and Equation (19) operate on the individual word units produced by the tokenizer described in Section 10. The same word units are used by the current IntentSpider graph and by the fixed next word evaluation data.

4.2 Personalized PageRank Formulation

The practical, on device version replaces the matrix exponential with a fixed point equation. Let $P = D^{-1}_{\text{transmission}} \cdot W_{transmission}$ be the row normalized transition matrix of the transmission type edge set. The personalized PageRank vector p satisfies

$$p = \alpha e_0 + (1-\alpha)P^T p \tag{5}$$

In this context, e_0 is the fan-dispersion-weighted seed from Equation (19), rather than an indicator vector on a single input word. The heat-kernel formulation in Equation (4) and the personalized PageRank formulation in Equation (5) therefore begin from the same IntentSpider seed construction, but they do not use the same diffusion kernel. Chung [4] analyzes the relationship between heat-kernel and PageRank diffusions; the two use different weighting series and should not be interpreted as one identical vector observed at different elapsed times.

The practical IntentSpider implementation uses the personalized PageRank/local-push formulation. The parameter $\alpha \in (0, 1)$ controls the restart strength and therefore the effective distance over which mass can spread before returning to the seed. PPNP and APPNP [16] are related because they also use personalized PageRank propagation, but in that work the propagated values come from a neural predictor. IntentSpider instead diffuses directly from recent word-node seed mass. Personalized web search [13] provides the broader PageRank personalization background.

In subsection 6.9.3 below, The IntentSpider system considers α as a fixed constant value which is visible above as well. In addition to that, Section 6.9.3 adds a second value which changes from the typing behavior. The raw keypress timing updates the sustained typing rate difference value. When the next word prediction is executed at the word boundary, this value can lower the effective restart value used by the diffusion. Meaning that, the signal can travel further while that sustained typing behavior remains active. The local push itself is executed at the word boundary in the WebAssembly process used by Section 10. This uses the separate symbols α_{base} and $\alpha_{effective}$.

4.3 The Local Push Process for Running the Diffusion Inside a Single Device

Solving this Equation 5 requires inverting an $|V| \times |V|$ system. meaning that, this requires a large processing amount if repeated for every next word prediction. In the WebAssembly process used by Section 10, character keypresses update the typing timing values and the committed word boundary triggers the graph update and prediction. This matters because $|V|$ can increase into over 10,000 or even a greater amount of words for a daily computer user across several years. In addition to that, Professor Reid Andersen and others (refer [1]) give an algorithm for this specific problem. This algorithm is named as "the local push." In this method, the process sets an approximate solution $\hat{p}$ and a residual vector r instead. And after that, it repeatedly pushes mass from any node which holds a residual value greater than ϵ to its nearby neighbor values. And this process ends once non of the residual values remain over that margin value. Their central result is a processing limiting value of

$$O\left(\frac{1}{\alpha\epsilon}\right) \tag{6}$$

And this limiting value is separate from $|V|$. It depends on the distance the signal needs to travel and on how precisely it needs to be resolved. This is the property this architecture uses for the local diffusion process. Section 9 gives the processing discussion for this point and Section 10 gives the measured result. The current working values use $\epsilon = 10^{-4}$, a maximum of 200,000 push operations for one prediction and a minimum $\alpha_{effective}$ value of 0.02. The processing amount therefore follows the local region reached by the signal instead of the complete amount of typing history stored for that human.

Section 6.9.3 adds the growing radius process to this processing relationship. When the keypress derived typing rate difference remains active, $\alpha_{effective}$ reduces and the diffusion can travel further. A lower α value increases the processing amount in the same bound. The current IntentSpider process applies this value when the next word prediction is requested at the word boundary. The minimum value of 0.02 limits how far this reduction can continue. (refer Section 6.9.3 and Section 9.2)

4.4 Ranking the Next Words and the Usage of the Entropy Signal

The next words are ranked by the relevant resulting values in p (or s(t)). And they are not ranked by a condition such as the specific proximity to the input. So, whether a word

was directly next to the input word is not used in here. In addition to that, there are two implications that happen afterwards from this process.

1. A word with no direct co occurrence history with the exact recent input sequence can still be a high ranked suggestion. This state is viable when that specific word is in an interconnected region area of the transmission type graph which the relevant seed reaches. Which means that, an exact recent word sequence can be new while the graph still reaches a related and already existing region through its other reliable connections.
2. The previously stated Shannon entropy of the resulting distribution, $H(p) = -\Sigma_v\ p_v \log p_v$, is a usable confidence signal. A "p" which is focused on a one intent depicts the intent values are more clear. And a p which is high in entropy depicts the intent is not clear. A normalized form of this output entropy is used later by the necessity gating process to decide whether the ranked next words are displayed. This is the first use of the entropy signal in IntentSpider. The second one is $\varphi(u)$ in Equation no. (18), which measures the fan out values of an input word. Section 4.1.1 also gives a separate possible use for the entropy of a longer token object. These are separate uses of the similar mathematical object on different distributions.

Section 4.5.8 below introduces a geometric option for the similar confidence question this specific entropy signal already answers. Meaning that, the distance from the embedding's centroid, and not the specific shape of the relevant distribution. And it is built from a separate tool in mathematics, which is the Laplacian eigenvectors. And that tool is not the entropy. Hence, it is not a fourth use of $H(\cdot)$. And also, it does not add a fourth point into the three above. Section 4.5.8 keeps these as two separate measurements of the same diffusion output: entropy describes how the value is spread, while the centroid distance describes where the probability-coefficient based average position lies in the geometric lens.

4.5 Geometric Paths, and a Two Dimensional Developer Embedding of the Intent Graph

4.5.1 The Embedding as a Two Dimensional Web

Let $L = D - W_{transmission}$ be the similar graph Laplacian which is already defined in the section 4.1. From the eigenvectors of its Laplacian, a graph's structure can be recovered in a low dimensional form. Professor Miroslav Fiedler (refer [30]) researched and found this first. And that research is for the single eigenvector which is tied to a graph laplacian's lowest eigenvalue which is not a zero value. (this is also known as the algebraic connectivity) In addition to that, Professor Mikhail Belkin and Professor Partha Niyogi (refer [31]) built this idea further into a usable low dimensional embedding. They did this process by solving the generalized eigenproblem $L f = \lambda D f$ for the eigenvectors ψ_1, ψ_2 which are tied to the two smallest eigenvalues $\lambda_1 \leq \lambda_2$ which are not zero values. And in here, the less complex or more trivial and constant eigenvector at the eigenvalue 0 is intentionally excluded. And after that, this method gives each node u the fixed coordinate

$$(x_u, y_u) = (\psi_1(u), \psi_2(u)) \tag{37}$$

And this coordinate can be computed again as the graph grows when new words appear. In the native developer process, the transmission graph is first converted into $W + W^T$, the largest connected component is used and the two non trivial directions are approximated through deflated power iteration. The developer terminal can rebuild this 2D embedding directly and also after a configured number of selection events. The WebAssembly playground

used for the Section 10 prediction measurement runs the prediction process separately from this embedding.

A personal typing graph can be a graph which is not connected. Which means, separate and unrelated clusters of the vocabulary, which that human never types near one another, is a thing that can happen. And even a likely one. And also, the Equation (37) considers this graph as a connected graph. A graph which is not connected has a zero eigenvalue with the multiplicity which is equal to its number of connected components. And it is not one. Hence, this changes which eigenvectors are considered as the two smallest values which are not zero values.

This is the specific part which makes the ”hexagon with the prey in the corners” spatial picture of this project true in a literal way for a one specific claim. And it is not only an analogy. It is a node which its transmission edges spread evenly has a high $\varphi(u)$ value. (refer Equation no. (18)) Therefore, specific node is pulled by many nearby neighbor values in many directions. Hence, it should remain closer to the embedding's centroid. And also, a node which its transmission edges focus on a few amounts of next words has a low $\varphi(u)$ value. This node is pulled in one direction by a few strong edges. Hence, it should remain in a separate region area. Hence, the $\varphi(u)$ value should correlate with $\|(x_u,\ y_u) - \text{centroid}\|$.

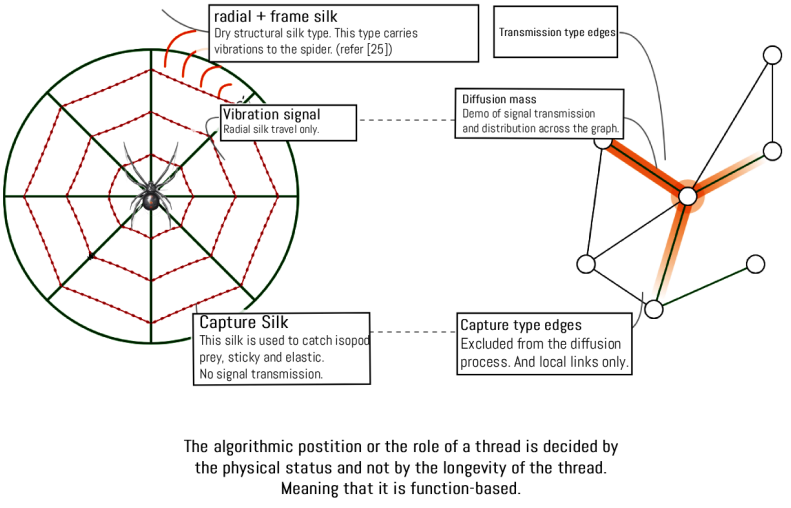

Figure 8. The structure of the spiderweb and the correlated position of an edge or a thread or embeddings inside the main intent graph.

4.5.2 Transforming a Sequence of Words into Paths

The section 4.5.1 gives a fixed coordinate for each node. In addition to that, a typed sentence gives a sequence of those next word distributions. In here, for the step i, define the position as the coefficient based average place of the diffusion output $\hat{p}_i$ across those coordinates.

$$\text{position}(i) = \sum_{v} \hat{p}_i(v) \cdot (x_v, y_v) \tag{38}$$

This is not only an analogy. T = (position(1), position(2), ..., position(n)) creates a path through a plane. In the current developer embedding process, one point is created for each successive word prefix whose diffusion output can be placed in the embedding. Meaning that,

this process creates a word prefix curve through that plane. The raw character keypresses are not separate points in this path.

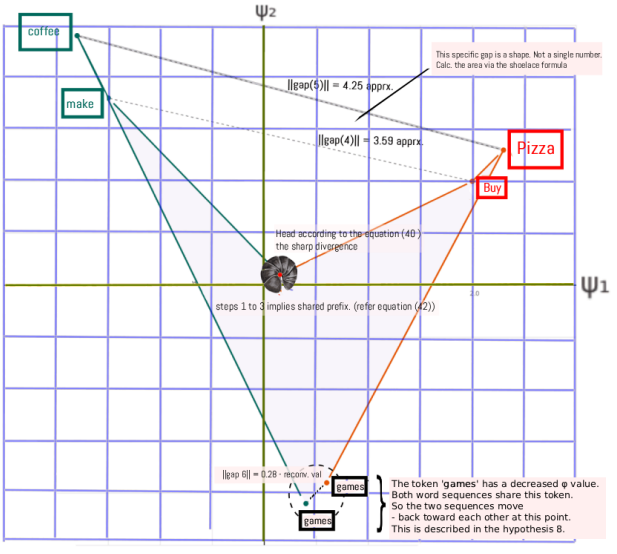

Figure 9. Paths produced by many sequences of next word distributions in the intent graph.

4.5.3 Theories related to Heads and Plateaus of the Curve

For the step i, the speed value of the path is defined as

$$\text{speed}(i) = \|\text{position}(i) - \text{position}(i-1)\| \tag{39}$$

A head is a local peak in the speed(i) value. And this peak is greater than the prominence margin value θ_{head}. Meaning that, the path moves more faster into a specific region area.

$$\text{head}(i) = \begin{cases} 1 & \text{if speed}(i) \text{ is a local maximum with speed}(i) - \max(\text{speed}(i-1), \text{speed}(i+1)) > \theta_{head} \\ 0 & \text{otherwise} \end{cases} \tag{40}$$

This head is expected to be in a relationship with a word with a low φ(u) value entering the recent seed window. The relationship described in Section 4.5.1 is applied here to one step of the path. In addition to that, a plateau is a run with minimum $n_{plateau}$ number of steps which happen one after the other. In that specific run, the speed(i) value remains lower than the tolerance value $\epsilon_{plateau}$.

$$\text{plateau}(i) = \begin{cases} 1 & \text{if speed}(j) < \epsilon_{plateau} \text{ for all } j \in \{i, i-1, \ldots, i-n_{plateau}+1\} \\ 0 & \text{otherwise} \end{cases} \tag{41}$$

And also, the plateaus are expected during a run of words with a high φ(u) value. Another scenario is when the path already remains near a well connected cluster. And the words after that only confirm it. In this method, the heads and plateaus picture receives a precise and a testable definition.

4.5.4 Two Paths in One Plane, the Shared Prefixes, and the Gap Between Them

If two sentences A and B share the exact similar word prefix with a length of k, their paths are the same over that prefix.

$$\text{position}_A(i) = \text{position}_B(i) \quad \text{for all } i \leq k \tag{42}$$

The native developer comparison gives Equation (42) a specific process. Its trajectory function runs the read only diffusion with the fixed α_{base} value while the compared word prefixes are created. It does not update reinforcement, TF, prey, shock or typing rate state during that comparison. Due to that reason, two equal word prefixes produce the equal path prefix when they are calculated from the same graph and the same developer settings. A separate live typing session can change the graph and the $\alpha_{effective}$ value before a later prediction. Equation (42) is the read only comparison from the shared graph and shared geometric settings.

After the two sentences separate, the gap at the step i is defined as a two dimensional vector.

$$\text{gap}(i) = \text{position}_A(i) - \text{position}_B(i) \tag{43}$$

The sequence of these gap values creates a region area. And this region has an area value. It is not a one single number for the separation. This area can be calculated directly using the ”shoelace formula” over the closed polygon created by both paths and their shared starting point. This is because a shape which opens and closes includes information which a one single number does not include. For an example, this depicts the way the two paths separate. (a wide and an early gap, compared to a narrow and a late gap which reaches the similar final distance) And it is not only the final distance value.

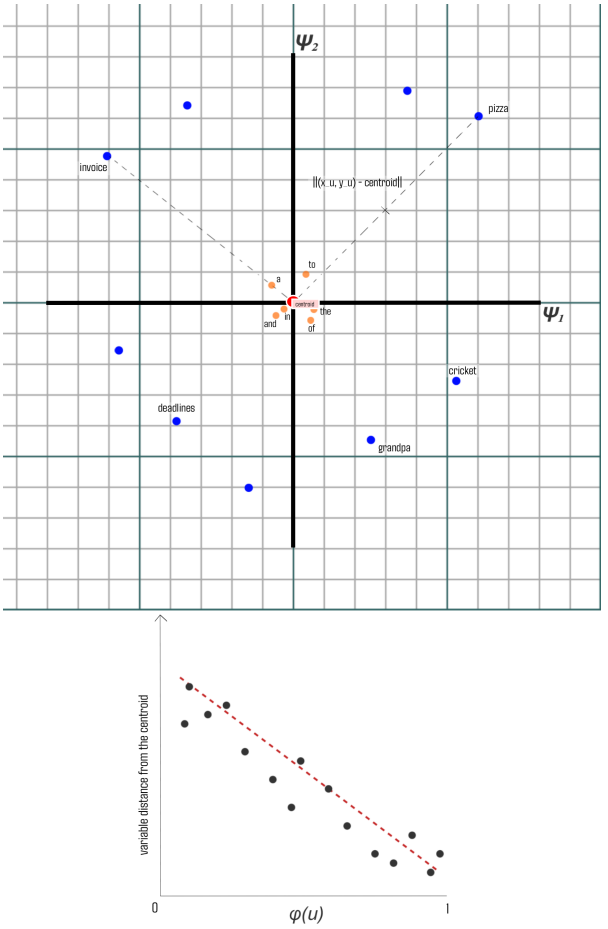

Figure 10. DualNet two path embedding and the corresponding gap behavior

4.5.5 The Reconvergence Process and Its Relationship With the Fan Dispersion Values

When both sentences share a word again later, whether the gap(i) value closes back to the value 0 is depending on that shared word's own φ(u) value. This is the most important point in this section. This is because it connects two previously separate parts of this paper into a one geometric explanation for a similar underlying idea. Those two parts are the spatial embedding in the section 4.5.1 and the fan dispersion coefficient process in the section 4.1.1.

1. A shared word with a high φ(u) value has a broad and a generic connectivity. Meaning that, it pulls both paths in a lower amount, and only to the centroid of the embedding. Hence, this is a weak and a generic closeness. And it is not a real reconvergence.
2. A shared word with a low φ(u) value has a narrow and a specific connectivity. It pulls both paths strongly into the similar distinct cluster. Hence, this is a real and a valuable reconvergence.

This is also similar to the points made in the section 4.5.1 about theories for the single paths. And in here, it is applied to two paths. Meaning that, words with a low φ(u) value carry and keep the position in a more strong way. Hence, a word with a low φ(u) value, which is shared by two separated paths, becomes true for both paths in the similar place.

And it closes the gap. A word with a high $\varphi(u)$ value carries the position in a lower amount. Due to that reason, sharing this type of a word closes the gap only in a small amount. Which means, only in the amount that any two separate points added to the similar centroid can close.

4.5.6 The Relationship With the Already Existing Entropy or Confidence Signal

The section 4.4 already gives a confidence signal to this architecture. This is $H(\hat{p})$, or the entropy value of the diffusion process output. And it is used directly in the necessity value in Equation (15). (refer section 7.3) In addition to that, the distance from the centroid (refer section 4.5.1) is a second and a geometry based signal. And it is calculated from the similar underlying object $\hat{p}$. These two signals should not be considered as a similar measurement in different types of forms. Which means that, those two are not similar and identical.

The entropy value measures how the probability mass is spread across the nodes. And this is separate from where those nodes are placed in the embedding. And also, the distance from the centroid measures where the probability coefficient based average place is. This is separate from how many nodes share that mass. Meaning that, a $\hat{p}$ value which is focused on a one node with a low $\varphi(u)$ value has a low entropy value. (which is a high confidence value in the section 4.4) And the geometric relationship in Section 4.5.1 places this kind of narrow connection farther from the centroid. This can be given as the common scenario. And in this scenario, the two signals move in the similar direction.

But a $\hat{p}$ value which is spread evenly between two nodes in opposite directions from the centroid has a high entropy value. (which is a low confidence value in the section 4.4) And its centroid can remain close to the center of the embedding. It is not near either of those two nodes. Meaning that, it is the average value of two points which are approximately opposite and away from the center. And also, a $\hat{p}$ value which is spread evenly between two nodes which are near each other in the embedding has the similar high entropy value. But its centroid remains where both nodes already are. Which means, away from the center, and it is not near the center of the embedding. Due to that reason, the two signals can differ from each other. And this happens because of the design choices and processes. It is not a measurement error.

4.5.7 A Working Example

Sentence A: "I want to buy pizza … games." And also, Sentence B: "I want to make coffee … games."

Step	word(s)	$position_A$(i)	$position_B$(i)	‖gap(i)‖
1–3	"I want to"	(0.15, 0.10)	(0.15, 0.10)	0 (exact, Eq. (42))
4	"buy" / "make"	(2.0, 1.0)	(–1.5, 1.8)	3.59
5	"pizza" / "coffee"	(2.3, 1.3)	(–1.8, 2.4)	4.25
6	"games" (shared, with a low $\varphi(u)$ value)	(0.6, –1.9)	(0.4, –2.1)	0.28

The gap value increases at the step 4. Meaning that, there is a head for both paths. (refer Eq. (40)) And the two paths separate into distinct and separate clusters. After that, this gap value increases more at the step 5. And after that, this value reduces in a large amount at the step 6. In that specific step, a shared word with a low $\varphi(u)$ value pulls both paths into the similar region area. Which means, this is the reconvergence process. And it is directly visible as a shape which is closing. (refer Section 4.5.5) And it is not calculated from a one number in the way the earlier version of this paper would have listed it.

4.5.8 Components of the Geometric Lens

Component	Role in the lens
Spectral embedding through Laplacian eigenvectors, centroid, Euclidean distance, and shoelace formula area (Section 4.5.1, Equation no. (37))	Read-only mathematical lens over the changing per-user graph [30, 31]
$\varphi(u)$ and distance from the centroid (Section 4.5.1)	Compares the graph-connection spread of a word with its place in the two dimensional lens
Centroid path for each sentence (Section 4.5.2, Eq. (38))	Places each word-prefix diffusion output into the lens and joins the positions into a path
Heads and plateaus (Section 4.5.3, Eqs. (39)–(41))	Marks local peaks and slower regions along that word-prefix path
Shared prefix exactness (Section 4.5.4, Eq. (42))	Gives the same read-only position when the token prefix, graph, and fixed lens parameters are the same
Gap as a shape between two paths (Section 4.5.4, Eq. (43))	Measures the separation between two word-prefix paths
Reconvergence and the shared word's $\varphi(u)$ value (Section 4.5.5)	Relates a shared word's connection spread to the way two paths move toward each other
Median line information content (Section 4.5.6, Eq. (44))	Gives a middle line between two paths for geometric inspection
Reverse or retrodictive diffusion (Section 4.5.7)	Describes reading the path structure in the reverse direction
Relationship with the entropy or confidence signal (Section 4.5.8)	Keeps the geometric position and the diffusion-distribution confidence as two distinct measurements

5. The Problem Related to the Blindness to the Valency

The architecture in Section 3 and Section 4 has a structural problem. This problem can be named as the blindness to the valency. equation (1), equation (16), and equation (17) do not include whether a selection event came from a text the human meant in a positive or a negative direction. The update process only includes that v was selected after u. And also, it includes whether that selection matched the output from the diffusion process. It does not include the type of the completed text.

A graph update process Φ on selection events (u, v, t) has the blindness to the valency if two selection events $e_1 = (u, v, t_1)$ and $e_2 = (u, v, t_2)$ come from texts with opposite emotional tone signals, and Φ produces the similar changes to (w, TF).

Proposition No. (1) The update process in equation (1), equation (16), and equation (17) has the blindness to the valency.

Related Proofs equation (1) includes only $\mathbb{1}$[selection event at t+Δt] and Δt. equation (16) includes only outcome(u, v, t). This ouput result is defined by whether (u, v) carried the diffusion mass toward the next word the person selected. Which means that, this is a result related to the success of the prediction. It is not a result related to the emotional tone of the completed text. equation (17) includes only whether TF(u, v, t) is greater than θ_{TF}. Non of these three equations include another value which represents the surrounding completed text. Due to that reason, changing the positive or the negative direction of that text does not change the output of these three equations. ■

This proof follows directly from the definitions. And this is the important point in here. The blindness to the valency is included inside the update process. It is not a failure which appears later from another process. Section 10 measures the signed and suppression channel by removing that channel under the same 500 event data. The largest measured change from that condition is the amount of ranked suggestions which reach the interface.

5.1 An Example With Two Selection Events

Consider a human who selects the next word pizza two times during several hours. The first text is "I need to buy _____." The second text is "I have had enough _____." Both of them are selection events on an edge which ends at pizza. The first selection should strengthen a connection related to buying pizza. But the second selection should suppress that connection. Which means that, in the second text, the human is closing the intent rather than asking for more pizza.

Under equation (1), both events add the similar reinforcement value $+\eta$ to the relevant edges. Under equation (16) and equation (17), both events can increase TF(u, v, t) toward the transmission type state when the diffusion process already produced that next word. The architecture does not include a process where the second event can add a negative or a suppression update. Confirming and closing texts can share the similar final next word in ordinary text. Requests, complaints, gratitude, and cancellations can be given as examples. Due to that reason, this is a main problem. It is not a problem which can be left for a later process.

5.2 Why the Existing Exhaustion Penalty Does Not Correct This Problem

The Tabu Search based exhaustion value in Section 3.2 can be considered for this problem. In the current IntentSpider process it is a short lived value which records how recently the relevant context to candidate connection was selected. Section 7 uses the greatest such value from the recent prediction context as a value which reduces the necessity value. This process does not change the stored word connection and it also does not separate the valency of those selection events.

Due to that reason, exhaustion can make a recently satisfied candidate less likely to pass the display gate, but it does not prevent that word from being ranked or transmitted through the graph. The same recency rule applies after both ”I need pizza” and ”I have had enough pizza.” And it does not change whether the relevant connection should be strengthened, weakened, or kept without another change for the future. The signed process in Section 6 changes a separate state on the word connection and can eventually place a sustained negative connection into $E_{suppressed}$. Exhaustion by itself does not correct the blindness to the valence.

---

6. Valence Gated Signed Diffusion

The Valence Gated Signed Diffusion part is a main invented contribution from this paper. It gives the IntentSpider graph a second signed state in addition to the ordinary raw connection value and transmission fidelity. Section 10 measures this signed and suppression channel by removing it while keeping the same 500 event evaluation data.

6.1 Design Goal

This process has three design goals. First, the timing trackers accept raw keyboard input continuously while next word prediction remains light enough to run locally at a committed word boundary. Second, a human's graph can continue learning from that human's own device data instead of requiring a population graph. Third, a selection event can push a word connection away from a next word rather than only stop strengthening it when the surrounding behavior indicates a closing or rejecting direction.

6.2 Scoring Valence On Device Through Typing Interval Patterns

Introduction. In spiders, especially the type "Wandering" Spider, the similar vibration event creates two behavioral responsesto opposite sides, which is based on a physical property of the relevent signal. A low frequency vibration between 3 and 4 (used base values for simplification) produces the approach and a fighting or a threat response And also, a greater vibration from approx. 350 to 450 value produces the escape response (wandering spider research paper. refer [26]). The spider does not need to read a meaning from that signal for this separation. Instead, the spider's CNS (or central nervous system) uses the physical shape of the signal. This physical-signal separation motivates the IntentSpider valence signal, which is built from how the human typed a selection rather than from a fixed emotional-word list. The timing signal therefore changes from one human and one typing period to another.

Typing timing is used here as a behavioral input rather than as a direct measurement of emotion. Prior work shows that keyboard typing has measurable temporal structure [32], while keystroke dynamics can also carry user-specific information [27]. Those findings motivate treating timing as a meaningful signal, but they do not validate the particular IntentSpider mapping below as a psychological measure of valence.

Definition. For a selection event at time t, Δt_{avg} represents the average time interval between the keypresses in the previous context window. Δt_{ref} represents the baseline typing speed of that same user. This is tracked for each human separately, rather than using one fixed constant for all humans. In addition to that, κ is the correction rate value in that similar context window. Which means the simple number of backspaces per each key pressing event. This can be defined as a valence value.

$$\mathrm{val}'(c_t) = \tanh\left(a \cdot (\Delta t_{avg} - \Delta t_{ref}) - b \cdot \kappa\right) \tag{20}$$

In this context, val'(c_t) is a signed value. The sign of this specific value is the main part used by the processes from subsections 6.3 to subsection 6.5. Below in here, subsection 6.9.1 adds a second unsigned value from the similar typing-rate information. It represents a size and not a direction and is used to construct IntentSpider's separate hexarousal axis. Professor Russell's term arousal remains the name of the research axis discussed in Section 2.8 and is not used here as a casual replacement for the IntentSpider signal. Section 6.9.1 builds a further group of processes from this value without changing val'(c_t).

6.3 Signed Edges

Every edge is extended to carry a signed value s(u, v, t) between –1 and 1, in addition to the existing raw coefficient value. For each selection event, this value updates as

$$s(u, v, t + \Delta t) = \text{clip}\big(s(u, v, t) \cdot e^{-\Delta t/\tau_v} + \eta \cdot \text{val}'(c_t) \cdot \mathbb{1}[\text{selection event}], -1, 1\big) \quad (7)$$

In here, c_t is the context window around the selection event at time t. And val'(c_t) is the typing interval and correction based signal defined in Equation no. 20. (refer Section 6.2). A confirming selection has a val'(c_t) value greater than 0. This pushes s(u, v, ·) into a positive direction. A closing or rejecting selection has a val'(c_t) value less than 0 and pushes this separate signed state into a negative direction. The ordinary raw coefficient in Equation (1) is still reinforced by the committed word in both cases. The signed value is a second and separate axis. A negative s value also builds the suppression history through Equation (21). After the suppression condition is reached, that connection contributes to $W_{suppressed}$. This is the correction process for the failure mode described in Section 5. Section 10 measures the output change created by removing this signed and suppression channel.

6.4 The Suppressive Set and Its Relationship With Transmission Fidelity

The two axes are separate by construction. TF(u, v, t) measures whether an edge reliably predicts the next word the human selects. Which means that, this value is about predictive success and it does not include emotional content (refer Proposition 1). s(u, v, t) measures whether the selection on that edge happened with a positive willness or a negetive willness, according to the behavioral timing process previously created. Due to that reason, an edge can have the transmission type and also build a negative valence value at that same time as well. This depicts a word connection which is predicted reliably and selected in a repeated way, but also selected with a reluctant behavioral signal. The pizza example in that previous section denotes this specific state. After "I have had enough pizza" is typed a sufficient amount of times, the diffusion process can, into a some extent relaibly produce the pizza again. But the typing interval patterns can depict that the human is closing the intent rather than asking for more.

The suppression rule, stated on the valence axis. An edge which repeatedly accumulates negative signed history can move into a third set named as $E_{suppressed}$. After that, the relevant word is not removed from the graph. Section 6.5 gives that connection a cancelling role inside the signed diffusion. Equation (21) represents the ordinary route as a buildup of the negative part of s(u,v,t) across earlier selection events. In the current system, this history is stored through an exponentially decayed negative accumulator. Due to that reason, the suppression state is separate from the TF based transmission and capture role, and it is created from the accumulated negative history.

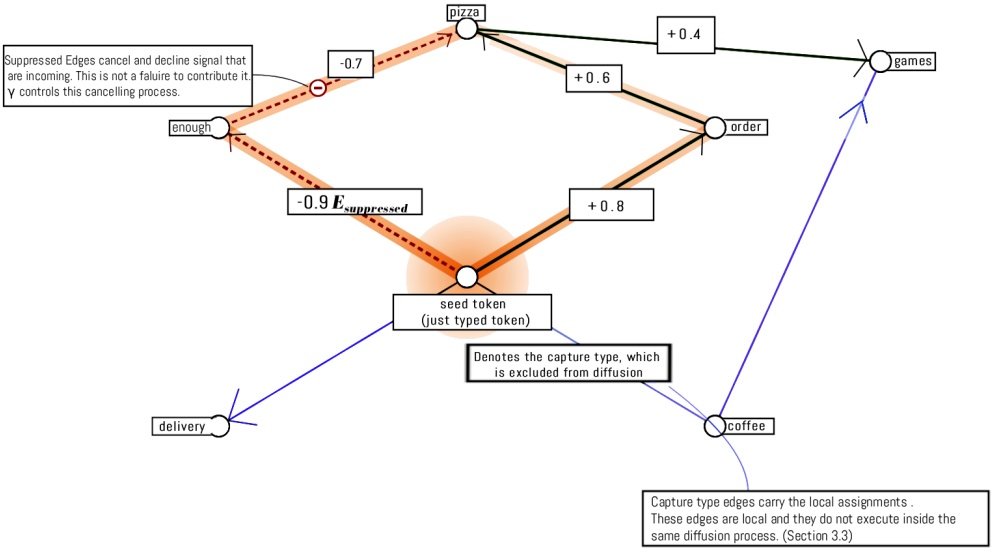

Figure 11. Transmission Type (TF) and Capture Type roles of edges, including the diffusion path and direct association connections.

$$\int_{t-T'}^{t} \max(0, -s(u, v, \tau))\, d\tau > \theta_s \implies (u, v) \text{ moves into } E_{suppressed} \tag{21}$$

Equation (21) gives the ordinary route into $E_{suppressed}$. It represents a slow buildup of negative valence across earlier selection events. In the current IntentSpider process, this value is stored as an exponentially decayed accumulator of $\max(0, -s)$ at the selection events. Section 6.9.4 adds a second and sudden route. In that route, one detected shock moves the connection into $E_{suppressed}$ immediately instead of waiting until the gradual value passes θ_s. The shock route also uses a slower decay value. Which means that, one route reaches the suppression set from gradual negative history and the second route reaches that similar set from one sudden event.

The three change speeds in this integration. Sections 3.2 and 3.3 built the TF based typing process with not using ratchet. An edge can move between the transmission type and the capture type as the TF value increases or lowers. The $E_{suppressed}$ process has a different purpose. It is slow and persistent, so a rejected word connection does not return to the ordinary state immediately after the human stops rejecting it. Due to that reason, the architecture includes one process created for fast reversal and another process created for slow persistence. An edge can be typed quickly as transmission or capture, and suppressed slowly at the similar time. Section 6.9.4 adds a third speed into this structure. A detected shock creates an immediate entry into $E_{suppressed}$. This exists alongside the fast TF reversal and the gradual negative value buildup from Equation (21). Which means that, a reader following how the separate system parts change should track three speeds and not only two.

$E_{suppressed}$ adds a persistent signed channel beside the freely reversible TF role. Which means that, predictive reliability, signed selection direction and persistent suppression can change at different speeds. Section 10 measures this channel directly. Removing it creates a large change in how often the ranked next words reach the interface.

6.5 Signed Diffusion Read Out

A negative edge should actively cancel a spreading signal. It should not only fail to add more signal. Due to that reason, the unsigned transmission type Laplacian $L = D - W_{transmission}$ from Equation (4) is replaced with a signed Laplacian. This signed Laplacian uses $W_{transmission}$ and $E_{suppressed}$ together. The process follows the signed spectral graph theory from Professor Jerome Kunegis and others (refer [18]).

$$L_{signed} = D_{|\cdot|} - W_{signed}, \qquad W_{signed} = W_{transmission} - \gamma \cdot W_{suppressed} \tag{9}$$

In this equation, $D_{|\cdot|}$ is the degree matrix built from the absolute edge coefficient values. This is required for the signed Laplacian. The γ value is greater than 0, and it controls how strongly a suppressed edge cancels the incoming signal at that node. The heat kernel and personalized PageRank read out processes from Section 4 remain available. Their seed comes from the fan dispersion construction in Section 4.1.1. In here, L_{signed} replaces L, and W_{signed} replaces $W_{transmission}$.

6.6 Algorithm

Algorithm 1

The equations in this section describe the graph update and signed diffusion parts. The result of a prediction becomes known when the next word is committed. Due to that reason, the current IntentSpider process uses the following order.

1. When a new word is committed, first resolve the previously pending prediction outcome and update transmission fidelity for the relevant prediction-carrying edges.
2. Read the accumulated typing interval and correction information for the committed selection and calculate $\mathrm{val}'(c_t)$ from Equation no. (20).
3. If a previous word exists, reinforce the ordinary raw coefficient through Equation no. (1), update the separate signed value through Equation no. (7), and update the ordinary negative-history suppression accumulator from Equation no. (21). A committed word therefore can strengthen the raw connection while its separate signed state moves in the negative direction.
4. Record the committed word and its previous context. The current TF value decides whether each edge has a transmission type or a capture type role through Equations no. (16)–(17).
5. At the next word-boundary prediction, build the fan-dispersion seed from the last k committed words and run the signed local push from Equation no. (9).
6. Rank the positive diffusion results after excluding the words used as the seed context. Calculate the entropy signal from the complete positive diffusion output.
7. Apply the optional residue and support choosing rule from Section 6.8 to the first and second ranked words. If fewer than three ranked words remain, the current system can fill an empty position from an unsuppressed direct association of the most recent word. That direct connection does not forward the diffusion again.
8. Store the resulting ranked row as the pending prediction. Section 7 then decides whether that row is displayed using the necessity gate. When the row is not displayed, those internal ranked words remain available for the next transmission fidelity outcome.

This list gives the core relationship between the function typed graph and the signed extension. Section 7 gives the complete evaluated runtime order, including raw keypress timing, compounding, the sudden negative response, the growing radius, the necessity gate, and the lazy settling event. This avoids treating a TF outcome as if it were available at the same instant the word being evaluated was first predicted.

6.7 The Local Push Mechanism State

Section 4.3 uses the local push algorithm from Professor Reid Andersen and others (citation no. [1]). The classical $O(1/(\alpha\epsilon))$ bound belongs to the non-negative setting. Equation (9) introduces signed cancellation, so the present paper treats the signed path through its implemented residual threshold and the measured processing results in Section 10 rather than restating the unsigned bound as a signed-graph theorem.

IntentSpider also allows the transmission and capture role to change when the committed next word updates TF. Section 10 gives the measured processing values from this signed and changing edge process.

6.8 Task Log From Entered Word History and Input Contexts

6.8.1 Introduction

6.9 History Based Task Scheduling From Previous Contexts

6.9.1 Introduction

Section 4 ranks every next word by using its diffused mass value. This is p or s(t) in Equations (5) and (4). But a separate choosing question remains after that process. The diffusion process can produce several next words at the similar time. Some of those words can include a already existing structure in the graph. This structure is named as "residue" or "residue values" in here. Another next word can be a new word and can result in a strong activation.

The system needs a process for selecting the most likely relevant word to suggest first. The valence processes from Sections 6 through 6.5 do not make this decision. They decide how the graph changes across selection events. The choosing process here is applied after the diffusion ranking is available for the current word-boundary prediction request.

6.9.2 Residue Value

The residue value is defined relative to θ_{TF}. For a candidate next word, the current process checks the recent context used by the prediction and keeps the greatest TF relationship found from that context. In here, the process is

$$r(v) = \min\left(\max_{u \in c_t : (u,v) \in E} \frac{TF(u, v, t)}{\theta_{TF}}, 1\right) \in [0, 1] \tag{22}$$

In here, $r(v) = 0$ means that no relevant recent context edge gives a positive TF value. The value reaches 1 once the strongest relevant TF reaches or passes θ_{TF}, and it remains limited at 1 above that point. The same value can therefore be calculated for a transmission type word after diffusion and for another candidate connected from the recent context. This value is used by the choosing rule below.

6.9.3 Support Value From Past Selection Events

The priority of a fresh next word is built from its previous selection events e_1, ..., e_n. Each event includes a time value named as t_i. And also, each event includes a context window c_i. This process defines two values for each one of those events.

$$R_i = \exp\left(-\frac{t - t_i}{\tau_0}\right) \text{ (PR Factor)}, \qquad S_i = \text{sim}(c_i, c_t) \in [0, 1] \text{ (PCS Factor)} \tag{11}$$

The PR Factor is knows as the Pure Recent Factor, and the PCS Factor equals the Pure Context Similarity Factor. R_i represents the recency of the event. A recent event produces a greater R_i value. And an older event produces a lower R_i value according to τ_0. S_i represents

the value of similarity between the earlier context c_i and the current context c_t. IntentSpider calculates this context relationship using Jaccard overlap between the two word sets. Those two values are separate and are not directly added. Instead, the system combines them through a logarithmic opinion pool. This is also named as a geometric opinion pool. It is an already existing process for combining two information sources through a one interpolation value. Professor C. Genest and others introduced this method (refer [7]). Another property of this process keeps the result consistent when the form of the parameters changes (refer [8]).

$$w_i(\rho) = R_i^{1-\rho} \cdot S_i^{\rho}, \quad \rho \in [0,1], \qquad \text{Support}_{\rho}(v,t) = \sum_i w_i(\rho) \tag{12}$$

A less complex process would be able to scale the recency constant directly by the value of context similarity. For an example, $\tau_{eff} = \tau_0(1 + \beta \cdot \text{sim})$. But this process does not produce the relevant end value. When τ_0 increases without a limit, each event's decay value increases toward 1. This happens without depending on its own value of similarity. Which means that, the pure context end value becomes a event count with equal coefficients. It does not become a context based value. Due to that reason, equation no. (12) uses the logarithmic pool. Its two end values can be checked directly. The two points below include those checks.

6.9.4 The Two End Values of the Interpolation Process

The first end value. $\text{Support}_0(v, t) = \Sigma_i R_i$. This is the Pure Recent value. And it does not depend on the context value.

Checking process. Put $\rho = 0$ into equation (12). Then $w_i(0) = R_i^1 \cdot S_i^0 = R_i$. ■

The second end value. $\text{Support}_1(v, t) = \Sigma_i S_i$. This is the Pure Context value. And it does not depend on the recent value.

Checking process. Put $\rho = 1$ into equation (12). Then $w_i(1) = R_i^0 \cdot S_i^1 = S_i$. ■

Those two end values were also checked by using numerical values. Which means that, they were not left only as algebra. The checking process used two created selection events. Event A is recent. In this event, t minus $t_i = 1$. But it has a lower context relationship, where $S_i = 0.10$. Event B is old. In this event, t minus $t_i = 50$. But it has a stronger context relationship, where $S_i = 0.90$. The process uses $\tau_0 = 10$. At $\rho = 0$, the Pure Recent total is 0.911575. This is exactly equal to $\Sigma_i R_i$. At $\rho = 1$, the Pure Context total is 1. This is exactly equal to $\Sigma_i S_i$. And these numerical values confirm the two exact end values.

Table 1 depicts another point which is not clear before checking the values between those two ends. At $\rho = 0.5$, Event A continues to have the greater contribution. At $\rho = 0.75$, Event B has the greater contribution. Which means that, the crossover happens between $\rho = 0.5$ and $\rho = 0.75$. In this specific example, the recent event needs a more lower coefficient before the context value receives the priority. This behavior comes from the geometric mean process. It is not a separate tuning decision. Due to that reason, $\rho = 0.5$ should not be read as an equal 50 and 50 balance in the practical system process. It continues to assign the priority to the recent event when the context relationship is not meeting the strength to the required level.

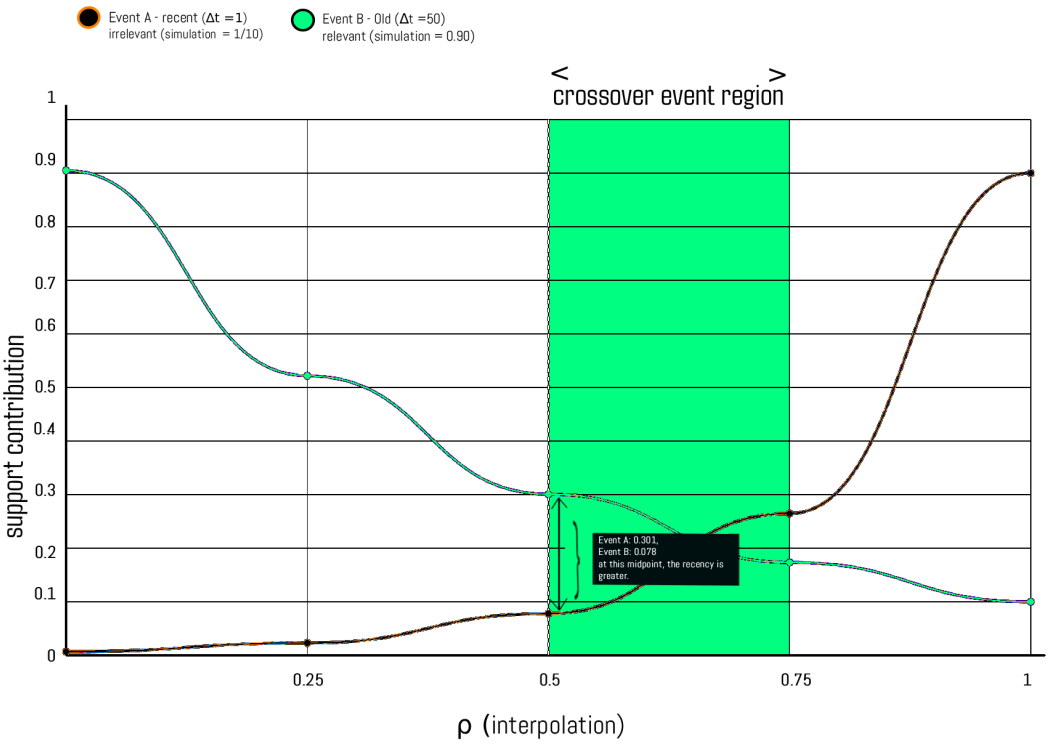

Figure 12. This figure shows the recent event and context based supporting behavior according to the value change.

Table 1. The contribution from each event to Support_ρ when ρ increases from the Pure Recent value ($\rho = 0$) to the Pure Context value ($\rho = 1$). The table uses the two created events above. The crossover between $\rho = 0.5$ and $\rho = 0.75$ does not happen at the midpoint.

ρ	Event A contribution	Event B contribution	Selected winner contribution
0.00	0.90484	0.00674	A
0.25	0.52171	0.02291	A
0.50	0.30081	0.07787	A
0.75	0.17344	0.26474	B
1.00	0.10000	0.90000	B

6.9.5 Selecting Between the First Two Ranked Words

The choosing rule is applied to the first and second words produced by the diffusion ranking. Let v_1 and v_2 be those two words. If $r(v_1) > 0$ and $r(v_2) < r(v_1)$, IntentSpider checks whether the second word has enough recent and context support to move above the first one.

$$\text{Support}_\rho(v_2, t) > c_{cal}\, r(v_1) \implies \text{swap}(v_1, v_2) \tag{13}$$

If those residue conditions are not true, the diffusion order remains unchanged. Due to that reason, this process can only exchange the first and second words. Support_ρ and r(v)

use two different value ranges. The process therefore includes the multiplier c_{cal} and the current working value is $c_{cal} = 1.0$.

6.9.6 The Working Values and the Controlled Result

Logarithmic opinion pooling is an established mathematical process. In IntentSpider it is used to combine the recent value and the previous context relationship before the optional choosing rule. The current working values use $\rho = 0.5$ and $c_{cal} = 1.0$.

Section 10 changes this choosing rule while keeping the remaining 500 event condition the same. The complete IntentSpider condition produced 8.4% first rank and 12.8% Top 3 accuracy. When this first and second word choosing process was removed, the result was 8.6% and 12.8%. Which means that, the measured aggregate accuracy change from this process was small in that fixed 500 event result.

6.10 The Typing Rate Difference Values and the Additional Runtime Processes

The processes in this section were designed from the attack and struggle analogy of the spider and the isopod prey described in the IntentSpider design history. (refer [38]) In IntentSpider, this analogy creates the processes for shared reinforcement, a growing diffusion radius, sudden suppression, settling and the repeating typing sub states.

Figure 13 denotes the wandering spider used in the vibration response research previously discussed in Section no. (2).

Figure 13. An American wandering spider type. (e.g.- Cupiennius Salei) The attacking behavior of this spider and the unique ability to respond to vibration motivated the design of the IntentSpider Webnet architecture.

The affective axis in subsection 6.9.1 and the communication channel point in the 6.9.8 are different from those six processes. Professor James Russell's research gives "valence" and "arousal" (These are described as valance and arousal axis and copied here as is.) as two dimensions of affect. Professor Russell's research keeps the term "arousal" for its second affective axis. IntentSpider deliberately uses the separate name "hexarousal" for its own typing-behavior based axis so those two objects are not treated as the same measurement. And Professor Joseph's research produces the communication channel difference previously discussed in the previous Section 2.8. (refer citation [29]) But Equation (23) only measures the delta of the typing rate, or the typing rate difference compared to the ordinary typing rate of that specific human. In a psychological perspective it also does not transform that typing rate into a psychological measurement value. IntentSpider Webnet's input is a typing behavior based value and it is a different point when compared to other researches.

6.10.1 The Typing Rate Difference Signal

Section 6.2 gives the valence signal val'(c_t). This signal gives whether the update process treats a specific selection as a confirmation result or a rejection result. But it does not give the amount that the current typing rate differs from the ordinary typing rate of that human. For an example, a slow reject and a faster reject with several corrections can both give a negative direction. The direction can be similar. But the amount of the typing rate change can be different. The correction rate is already denoted with κ in the (20) equation as well. The typing rate difference does not have a separate value in that equation.

Professor James Russell's circumplex model denotes valence and arousal as two separate dimensions (refer section 2.8 and citation no. [28]). The first dimension gives the pleasant to unpleasant direction. The second one gives the activated state to deactivated state dimension. As scientists conducted research related to this two dimensional method earlier, this foundation is not invented or novel. But a typing rate value does not become a measuring unit of Professor Russell's axis related to the arousal values, because the value increases. IntentSpider names this second system axis as hexarousal axis. And it names the value as noted below according to the typing data, which creates that axis overtime.

Let x(t) represent the typing rate calculated through a short trailing window ending at t. And let $\mu_{user}(t)$ and $\sigma_{user}(t)$ represent the rolling mean and rolling standard deviation of x(t) for that specific human. These values create a rolling baseline for that human. In the current IntentSpider process, the current typing rate enters the exponential moving mean and variance with a small update coefficient before the TRD value is calculated. Which means that, the current observation can move its own baseline by a small amount while the baseline still remains a per user value. In here, define

$$\mathrm{TRD}(t) = \mathrm{clip}\left(\frac{|x(t) - \mu_{user}(t)|}{\max(\sigma_{user}(t), \epsilon_{rate})},\ 0,\ \mathrm{cap}\right) \tag{23}$$

TRD(t) type of notations refer to the standardized typing rate difference. ϵ_{rate} represents a small positive value. This postive value is important. It has two purposes. First, it is the value that prevents diving by 0, which results "undefined x y" errors in C++. and second is when the earlier typing rate has no measured difference across its values. Equation (23) uses an absolute modulus difference. Due to that reason, TRD(t) has no positive or negative directions. A period faster than the baseline level and a period slower than the baseline level can give a similar TRD(t) value. The sign of $x(t) - \mu_{user}(t)$ keeps the faster or slower typing direction in places where another process is actively requesting it (or using it). And the sign of val'(c_t) keeps the separate confirmation or rejection direction and it is used by the signed update process.

This separation is required. TRD(t) does not prove a psychological state of the human. But rather, it is distance the current typing rate is from the ordinary typing rate of that specific human. Corrections, pauses, a device change, a physical interruption, or a deliberate change in typing speed can create that difference. For further details, in the subsection 6.9.6 denotes this loss of information. It keeps the separate typing direction inside a larger feature vector.

Δt_{ref} in Section 6.2 and the $\mu_{user}(t)$ and $\sigma_{user}(t)$ values in this section follow a similar per user structure. Non of them use a population typing margin value. A population value can read one human incorrectly when the ordinary typing rate of that human differs from the population value. IntentSpider's prediction engine can calculate these values locally. (refer Section 3.1) The Webnet playground also includes a separate state saving process when that deployment option is used.

6.10.2 The Shared Reinforcement Process Across Recently Selected Word Connections

Introduction Equation (1) reinforces one edge after each selection event. But several other next word edges can remain in the active state during a similar short period. The motivation in here is the analogy of a spider receiving continuous and multiple vibration signals from a struggling prey across several threads, rather than receiving one separate strike. Here in IntentSpider system that specific analogy and process becomes a shared and clustered reinforcement process across the edges (connections between words) which were active recently.

Definition. Let P(t) be the active prey set. In the current IntentSpider process, this set is created from the word connections which received a selection event during the short trailing window w_{active} before t. A diffusion value without a selection event does not add that edge into P(t).

$$P(t) = \{(u, v) : \text{a selection event on } (u, v) \text{ occurred during the final } w_{active} \text{ period before } t\} \tag{24}$$

With the number of edges in P(t), the diffusion budget amount increases. Which means this is a dynamic value.

$$B(t) = \min\big(\eta_{cluster} \cdot (1 + \delta)^{|P(t)|-1},\, B_{max}\big) \tag{25}$$

In here, $\eta_{cluster} > 0$ represents the base reinforcement value. $\delta \geq 0$ represents the increase added using the number of active edges. And B_{max} is a value that is assigned to the maximum value for the diffusion budget. When P(t) includes one edge, the budget is $\eta_{cluster}$. When the number of edges increases, the budget also increases until it reaches B_{max}. Which means the equation no (25) allocates a greater reinforcement during a period with several qualifying active edges.

The system shares this budget across P(t). Each edge receives one part based on its diffusion mass with values which are not negative values.

$$\Delta w(u, v, t) = \frac{B(t) \cdot \hat{p}(v)}{\sum_{(u',v') \in P(t)} \hat{p}(v')}, \quad \text{for each } (u, v) \in P(t) \tag{26}$$

Equation (26) shares the reinforcement according to the diffusion mass. When a recently selected edge does not have a positive value in the most recent diffusion result, IntentSpider uses the small non trivial mass value already used by the engine for this calculation. The shared update is executed when P(t) includes more than one selected active edge. This value is added together with the ordinary reinforcement in Equation (1).

This is a small note about notations in this section.

1. $\eta_{cluster}$ used for base reinforcement value in equation (25) This is a separate symbol than β_1, β_2, and β_3 in the section 7,3 for necessity gating functions.

2. this is different than η, which the equation no. (1) using as learning rate. This should not be confused.

3. $\eta_{cluster}$ is similar to the pattern η_{TF} in the subsection 3.2.

4. The δ and B_{max} symbols are separate from other values in the paper as well.

6.10.3 Increasing the Diffusion Radius Under a Sustained Typing Rate Difference

The motivation in here is the analogy of a spider changing its reach settings when the isopod is in an active mode. IntentSpider separates two values in this process. TRD(t) from Equation (23) is the current unsigned size of the typing rate difference relative to the rolling baseline of that human. A short burst or a long pause can give a greater TRD(t) value. A second value, $\ell(t_i)$ below, counts the consecutive keypresses where that typing rate difference remains

greater than a margin. The growing radius process uses this sustained streak for changing the later word boundary diffusion radius. These values describe the measured typing behavior. They do not imply a neurological or psychological measurement of that human.

Definition. Let t_i represent the time of the current keypress. Let θ_D represent the typing rate difference margin value. And let $\ell(t_i)$ represent the number of consecutive keypresses ending at t_i where TRD remains greater than θ_D.

$$\ell(t_i) = \max\{\, n : \mathrm{TRD}(t_{i-j}) > \theta_D \text{ for every } j = 0, \ldots, n-1 \,\} \tag{27}$$

This definition uses the actual keypress times t_i. It does not subtract an integer from a continuous time value. After that, the following function gives the gradual increase from this sustained period length.

$$\zeta(\ell) = 1 - e^{-\ell / L_0}, \qquad \zeta(\ell) \in [0, 1) \tag{28}$$

The function starts from 0. It moves to 1 when the number of consecutive keypresses increases. But it does not reach 1 for a finite ℓ value. The effective teleport value used in Eq. (5) then becomes

$$\alpha_{effective}(t_i) = \alpha_{base} \cdot (1 - \zeta(\ell(t_i))) \tag{29}$$

Every raw keyboard stroke updates the typing-rate tracker which can change the current sustained-period value. The evaluated WebAssembly engine then reads the resulting $\alpha_{effective}$ when the next word prediction is executed at a committed word boundary, and uses that value in the local push instead of the ordinary α. α_{base} represents the base value used when the sustained typing-rate-difference period is not active.

There is also a collision with the notation ρ. In Equation no. (12), ρ already represents the interpolation between the recent value and the context value in the logarithmic geometric opinion pool. Due to that reason, this section uses $\zeta(\ell)$ for Equation no. (28). And it uses θ_D instead of θ_A, because the process starts from the typing rate difference and not directly from Professor Russell's arousal axis. Inside the program configuration, this same working margin keeps the engineering name `theta_a`. In this paper the symbol θ_D keeps it separate from Professor Russell's arousal value.

The α value controls how frequently the diffusion returns to the starting point (refer Section 4.2). A lower α value lets the signal travel further before that return. When $\ell(t_i)$ increases, $\zeta(\ell)$ increases and $\alpha_{effective}(t_i)$ reduces. Which means that, the diffusion distance increases gradually while the greater typing rate difference is continuing. Hence the direction of the equation matches the growing radius value.

In Equation (29), $\alpha_{effective}$ reduces when the sustained period becomes longer. The processing amount in the $O(1/(\alpha\epsilon))$ expression increases when α reduces. The current implementation uses $\alpha_{min} = 0.02$, which keeps the restart value bounded while allowing the diffusion radius to expand. We also add the human typing behavior as an input to the diffusion process. The shared prefix proposition in Section 4.5.4 considers the words and the graph as the inputs. In the equation (29), it adds the history of the keypress timing. As in the section 4.5.4, due to that reason, two prefixes with the same words are not allocated to output a similar diffusion output when a human typed those words through different different timing patterns.

Section 4.3 keeps two questions about the $O(1/(\alpha\epsilon))$ bound. The first one comes from signed edge values. The second one comes because committed selection outcomes can change which edges are in the transmission type set between prediction requests. Those two points ask whether local push finalizes and ends in the same bounded form under those changed

graph conditions. And the process in this subsection adds a third and separate type of disadvantage. It does not change the α value into an unusable value during one finite keypress period. Instead, it deliberately reduces the α value used by the bound. And the processing amount increases when that value reduces. Due to that reason, this is a processing disadvantage. And this disadvantage is denoted in the section 9.2 as well.

6.10.4 The Sudden Negative Selection Response and the Suppression Process

The motivation in here is the analogy of a physically struggling isopod prey, which makes the spider travel from one specific point to the other and remain within that point for a short period of time. The first response is related to the specific edge connected with that event. The second response changes a specific value in the system for a limited amount of time. Here in IntentSpider system, this specific analogy and process transforms into a pattern taken from val'(c_t), which gives whether the update process reads a specific selection as a confirmation result or a rejection result, and the correction rate, which is already denoted with κ in the (20) equation. But the detected event does not prove that the human experienced a psychological shock. It only gives that pattern from those two values, and it does not necessarily imply IntentSpider identified a neurological or a psychological information of the human.

Let $\mu_{val,user}(t)$ and $\sigma_{val,user}(t)$ represent the rolling mean and standard deviation of the earlier val'(c_t) values for that specific human. The system does not include the current value in this baseline. Let $\epsilon_{val} > 0$ represent a small positive value. This value prevents the standard deviation from becoming 0 in the margin value calculation. The system detects the event when val'(c_t) is lower than the earlier mean of that human by more than k standard deviations

$$\text{shock}(t) = \begin{cases} 1 & \text{if val}'(c_t) < \mu_{val,user}(t) - k \cdot \max(\sigma_{val,user}(t), \epsilon_{val}) \\ & \text{and } \kappa(t) > \kappa_{burst\_marginvalue} \\ 0 & \text{otherwise} \end{cases} \tag{30}$$

Equation (30) detects an unusual negative value in the statistical values of that human, together with a correction burst. It does not identify the reason which created that pattern. An ordinary typing error, a physical interruption, a device change, or typing with frustration can activate it. Hence, the equation gives an sudden negative selection response inside the system.

Local response. When shock(t) equals 1 during a selection event on edge (u, v), the system moves that specific edge into $E_{suppressed}$ immediately. It does not wait until the negative integral in Eq. (21) passes θ_s.

$$\text{shock}(t) = 1 \text{ on } (u, v) \implies (u, v) \in E_{suppressed}, \text{ with decay constant } \tau_{shock} \tag{31}$$

In here, τ_{shock} represents the persistence period for the sudden route into $E_{suppressed}$. The current implementation uses $\tau_{supp} = 604800$ seconds, approximately seven days, for an ordinary Equation (21) entry and $\tau_{shock} = 2592000$ seconds, approximately thirty days, for a shock entry. Both routes enter with the same initial suppression strength, while the shock-marked state decays more slowly.

During a cool down window w_{shock} after the detected event, the system increases the necessity gating margin value in Eq. (15). Section 7.3 owns that margin value. And it gives this temporary change as Eq. (36).

This corrects the earlier equation direction. Equation (15) displays a suggestion when N(w, t) is greater than κ_N. Reducing κ_N makes that requirement easier to pass. Which

means that, the system can display more suggestions. A temporary response which is more careful needs the opposite direction. Due to that reason, the system increases κ_N. This makes the requirement more difficult to pass.

This is a small note about notations in this section.

1. This margin value is κ_N. It is not τ_N.

2. The τ symbols in this paper represent decay constants. This includes τ_v, τ_x, τ_0, and τ_{seed}.

3. The necessity margin value is not a decay constant. This subsection describes the relationship between the necessity gating algorithm and the overriding process. The local response and the system wide response give a specific form to an earlier design point. That design point was a second signal which can return or cancel a diffusion output with a greater confidence from being displayed. The necessity gate in Section 7 already gives the continuing event executor by using the process denoted there. In addition to that, equation no. (30) adds a separate event event executor. The system does not add the continuing gate and the sudden executor into a one score. The event changes the already existing margin value for a limited amount of time. Then the equation no. (31) is executed to change that edges' state

6.10.5 The Grace Period and How Temporary Changes Return to Their Ordinary State

This specific mechanism is inspired by a behavior of an animal predator (Orb Spider). In this scenario, the elevated responsiveness factor reduces after the prey stops its physical activity. In IntentSpider, the active period ends after the grace time passes without a new selected edge keeping that period active. Equation (32) separates a confirmed ending and an unconfirmed ending. A confirmed period uses $m_q = 1$ and keeps the ordinary decay rate. An unconfirmed period uses $m_{unconf} > 1$. Due to that reason, the temporary compounding value returns more faster after an unconfirmed ending.

The definition of this process is, if q represents a single period of activity, then if t_q^{end} denote the time of its last and final activation that is greater than or equal to the specific limit A settling event starts when none of the edges in P(t) receives another activation that is greater than or equal to that limit during T_{grace}. The default value used in here is 60 seconds, or a complete minute.

Let $\Delta w_q(u, v)$ represent only the coefficient change added by the equation no. (26) during the period, which is q.

For the second line below, let $\Delta\alpha_q(t') = \alpha_{base} - \alpha_{effective}(t')$ represent only the teleport reduction related to that period. Equation (32) describes its return toward the ordinary state. In the current IntentSpider process, the stored temporary values are the compounding coefficient differences. At the settling event, the typing rate streak which creates $\alpha_{effective}$ is reset back to its ordinary state.

A confirmed period is one where val'(c_t) is greater than 0 at the final selection event. Otherwise, the period is unconfirmed. Define $m_q = 1$ for a confirmed period. And define $m_q = m_{unconf} > 1$ for an unconfirmed period.

For $t \geq t_q^{end} + T_{grace}$, the return function process is

$$\begin{aligned} \Delta w_q(u, v, t) &= \Delta w_q(u, v, t_q^{end}) \cdot e^{-(t - t_q^{end} - T_{grace}) m_q / \tau_v}, \\ \Delta\alpha_q(t) &= \Delta\alpha_q(t_q^{end}) \cdot e^{-(t - t_q^{end} - T_{grace}) m_q / \tau_\alpha}. \end{aligned} \tag{32}$$

Equation (32) gives the return of the temporary values. IntentSpider stores the individual compounding differences added during the active period. The settling check runs at the first later system event after T_{grace} has passed. When the final selection valence is not positive,

the remaining compounding difference follows the faster m_{unconf} decay. A confirmed period keeps the ordinary τ_v decay. After this process, the active prey set is cleared.

For the radius value, the current IntentSpider process resets the sustained typing rate streak at the settling event. Which means that, the next $\alpha_{effective}$ calculation returns to the base condition. Equation (32) keeps the separate τ_α form as the mathematical description of a gradual radius return, while the current process uses the streak reset.

6.10.6 Discovering Per User Typing Sub States From Measured Typing Patterns

Introduction. TRD(t) is a one single value and it has no direction. Meaning that, its a scalar quantity. It does not keep whether the typing becomes more faster or more slower. And a similar value can appear with a different correction rate, pause structure, or val'(c_t) direction. For an example, fast typing with a lower number of corrections, fast typing with greater number of corrections, and a long paused state after a selection are separate typing states. Equation (23) can place them under a similar magnitude. After that, we keep those measured patterns as typing sub states as well.

Let

$$f(t) = \big(z_x(t),\ \mathrm{TRD}(t),\ \widetilde{\kappa}(t),\ g_{pause}(t),\ \mathrm{val}'(c_t)\big), \qquad z_x(t) = \frac{x(t) - \mu_{user}(t)}{\max(\sigma_{user}(t), \epsilon_{rate})} \tag{33}$$

In here, $z_x(t)$ keeps the faster or slower typing direction. And TRD(t) keeps the limited absolute magnitude from Equation (23). $\widetilde{\kappa}(t)$ represents the correction rate after a per user normalization. $g_{pause}(t)$ represents the pause pattern feature. And val'(c_t) is the selection direction already used in the previous section no. 6.2. These separate parts need comparable scaling before one distance margin value combines them. Otherwise, one part with a greater numeric unit can decide the cluster distance by itself.

An online clustering process can run per user and inside the device IntentSpider is running. It groups the recent f(t) values using a distance margin value $\delta_{cluster}$. (The number of groups is not a fixed value before the typing data exists). And the meanings of those groups are also limited to the patterns in the measured typing features. But this equation (33) is not a definitive output of the human user's emotional state, and it alone cannot name one group as happy state, sad state, rushed state, frustrations or another emotional state.

The current grouping process uses the typing rate direction, the absolute typing rate difference, the correction rate and the valence value. It uses the nearest existing center from the earlier groups. When that center is farther than $\delta_{cluster}$, the process creates a new group center. Otherwise, the earlier center moves through its running mean. This process keeps a maximum number of groups and creates an observational description of the measured typing pattern.

6.10.7 Combining Several Typing Vector Values Into One Behavioral Value

Sections 6.9.2 and 6.9.3 use a typing rate difference without a direction. Equation (33) keeps more than one measured behavior. This subsection gives a one bounded value from those features for a place where the system needs a multiplier with values which are not negative values. This value is a behavioral interaction coefficient. It is not a direct measurement of a psychological state.

Let $\widetilde{g}_{pause}(t)$ represent a pause feature normalized into a value from 0 to 1. Let $\widetilde{\kappa}(t)$ also remain from 0 to 1. And let $\widetilde{\mathrm{TRD}}(t) = \mathrm{TRD}(t)/\mathrm{cap}$ when cap is greater than 0. In here, define

$$\xi(t) = \mathrm{clip}\Big(\tanh\big(\nu_1 \widetilde{\mathrm{TRD}}(t) + \nu_2 \widetilde{\kappa}(t) + \nu_3 \widetilde{g}_{pause}(t) + \nu_4 |\mathrm{val}'(c_t)|\big),\, 0,\, 1\Big) \tag{34}$$

The ν_1 to ν_4 coefficients have values which are not negative values. Equation (34) therefore gives 0 when all four input magnitudes are 0. And it moves to 1 when their coefficient based combination increases. The absolute value of val'(c_t) keeps the confirmation or rejection direction outside this magnitude. This is required if $\xi(t)$ replaces another multiplier which cannot use a negative value. If the system uses val'(c_t) with its sign inside the sum, a negative selection can reduce or reverse the coefficient. That would create a separate process.

This is a note about notations in this section.

1. This value is $\xi(t)$. It is not $\alpha_{coefficient}(t)$. The α symbols in this paper represent the teleporting process. This includes α, α_{base}, and $\alpha_{effective}$. They should not be confused and please note that we use $\xi(t)$ for the equation (34).

2. There is a collision with the notation w. This paper uses w(u,v,t) for edge coefficients. And in the equation no. (12), it already uses $w_i(\rho)$ for the logarithmic pool. Section 7.3 uses β_1, β_2, and β_3 for necessity gating functions as well. They should not be confused. And we use ν_1 to ν_4 for the coefficients in the equation (34).

The earlier coefficient names w_1 to w_4 also connect with notation already used in this paper. This paper uses w(u,v,t) for edge coefficients. And the equation no. (12) uses $w_i(\rho)$ for the logarithmic pool. And section 7.3 uses β_1, β_2, and β_3 for necessity gating algorithm.

In addition, separately, coefficients in Eq. (34) are named ν_1 to ν_4.

6.10.8 Calibrating Typing Rate Difference Values Across Communication Channels

The difference between the communication channels does not come from the analogy of the spider. Professor Joseph Walther's "Hyperpersonal Model" (refer [29]), together with the cues filtered out and social information processing research discussed in Section 2.8, showed that computer mediated communication is different from face to face communication. And this research establishes something very specific. Meaning that, how the affective information appears and how it is read is depending on the specific medium of the communication used. Text messaging, email, note taking, and another written communication system do not give the human the same typing conditions.

The implication for this system is a limited one. A typing rate baseline calculated inside one communication channel should not be used directly for another communication channel. The ordinary typing rate and the pause behavior of the same human can change based on the task and the communication system. Let $\mu_{user,channel}(t)$ and $\sigma_{user,channel}(t)$ represent the earlier baseline values calculated for one human inside one communication channel. One calibrated value is

$$\mathrm{TRD}_{channel}(t) = c_{channel} \cdot \mathrm{clip}\left(\frac{|x_{channel}(t) - \mu_{user,channel}(t)|}{\max(\sigma_{user,channel}(t), \epsilon_{rate})},\, 0,\, \mathrm{cap}\right), \qquad c_{channel} > 0 \tag{35}$$

In here, $c_{channel}$ represents a calibration value calculated from data for that specific communication channel. A baseline calculated separately for each channel already removes one part of the ordinary difference between the channels, so $c_{channel}$ represents the remaining measured channel difference.

Equation (35) only gives a channel calibrated typing rate difference value. It does not change $\mathrm{TRD}_{channel}(t)$ into the affective axis or hexarousal. The affective axis is a separate system process. It is a value which moves to higher levels as the user's typing remains in a sustained engagement. Equation (35) does not produce this movement by itself. A separate

process needs to use the typing rate difference values across several keypresses before the system can make that statement. Professor Russell's circumplex model keeps "arousal" as the name of Professor Russell's own axis (refer [28]). That name does not replace the name of the IntentSpider signal.

This equation also does not claim one fixed behavior when the human moves between communication channels. It gives a warning that the system needs separate data for each channel. If one channel does not include a sufficient amount of earlier typing data, the system cannot state that its baseline for that channel is calibrated. In that state, the system can use a per user baseline which is dynamically calculated based on the huamn's typing patterns.

---

7. Necessity Gating Algorithm and Suggesting Functions

The processes from Section 3 to Section 6 decide which next words receive the highest ranking. After that process, there is another decision about whether those ranked words should be displayed to the human. Necessity gating gives IntentSpider this second decision. It uses values already created from the graph, the prediction distribution and the recent selection history. Section 10 measures this process by removing the necessity gate while keeping the remaining prediction process.

7.1 Using the SCAMPER Method to Generate This Process and Obtain Results

The process below was generated using SCAMPER, or Substitute, Combine, Adapt, Modify, Put to another use, Eliminate, and Reverse. SCAMPER is a checklist based design method built from the research work of Alex Osborn and the later SCAMPER formulation. (refer [23]) (refer [24]). In IntentSpider, this process creates a separate value for deciding whether the already ranked next words should be displayed.

- Substitute. Replace the implicit rule which always displays the first-ranked next word with a continuing necessity value and a margin value.
- Combine. Combine the confidence value obtained from the prediction distribution, the valence-based urgency value, and the recent exhaustion value already produced by the architecture.
- Adapt. Use the exhaustion value in Section 3.2, Equation no. (14), as a term which reduces the necessity value when a recently selected connection is repeated again.
- Modify and Magnify. Keep the margin as a configurable quantity, so the display requirement can be made more or less selective and can later be calibrated from earlier data of one human.
- Put to another use. Assign the necessity value to the display decision. The graph-learning processes remain with the raw, transmission-fidelity, signed-valence, suppression and compounding updates described in the earlier sections.
- Eliminate. Remove the gate and return the ranked row whenever candidates exist. Section 10 uses this condition to measure the effect of the ordinary necessity gate.
- Reverse. The reverse display state is silence: when the value stays below the margin, the ranked row remains internal and the interface displays no next-word suggestion.

7.2 The Necessity Value

Let w represent one next word at time t. The necessity value combines three values which the architecture already produces.

$$N(w,t) = \beta_1 \cdot (1 - H_{norm}(p)) + \beta_2 \cdot \max(0, \text{valence}_{urgency}(c_t)) - \beta_3 \cdot \text{exhaustion}(w,t) \quad (15)$$

Two quantities in Equation no. (15) use the following definitions.

- Normalized confidence. Let the positive part of the local push output contain m nodes with values greater than 0. IntentSpider normalizes those positive diffusion values while calculating Shannon entropy. It defines $H_{norm}(p) = H(p)/\log(m)$ when $m > 1$. When $m \leq 1$, the value is 0. In here, m is the number of positive nodes in the diffusion output. The one hop capture type fallback is added after this entropy value is calculated. Which means that, Equation no. (15) receives the confidence value from the greater positive diffusion output even though the interface returns a maximum of three next words.

- Valence urgency. Define $\text{valence}_{urgency}(c_t) = \max(0, -\text{val}'(c_t))$, where $\text{val}'(c_t) \in [-1, 1]$ is the typing interval and correction based context valence from Section 6.2, Equation no. (20). At a committed word boundary, that typing interval window first gives its value to the signed update for the selected connection. After that, the window is closed before the next word prediction. Due to that reason, the necessity calculation immediately after that word boundary receives the neutral urgency value in the current system process.

The $\text{exhaustion}(w, t)$ value comes from Equation no. (14) in Section 3.2. The current process checks the recent context words used to seed the prediction and keeps the greatest exhaustion value among their edges into the current first-ranked word:

$$\text{exhaustion}(w, t) = \max_{u \in c_t : (u,w) \in E} \text{exhaustion}((u, w), t).$$

This keeps the exhaustion term local to the same recent context used by the prediction.

β_1, β_2, and β_3 are the three coefficients. κ_N is the display margin. IntentSpider uses the assigned values of these parameters during prediction. Section 10 measures the effect of this margin by removing the gate while keeping the remaining prediction process. In that process, removing the gate increased the number of displayed suggestions and also exposed additional exact next word hits. Which means that, the current margin is an important value for the later calibration of this process.

The notation uses κ_N because τ is already used by decay constants such as τ_v, τ_0, τ_x, and τ_{seed}, while θ_{TF} and θ_s denote other graph margins. The coefficients use β_1, β_2, and β_3 because w already denotes the raw edge coefficient and $w_i(\rho)$ is already used in Equation no. (12).

The system displays a suggestion when $N(w, t) > \kappa_N$. When the value is equal to or lower than the margin, the ranked row remains internal and the visible suggestion row is empty.

The shock retreat process in Section 6.9.4 changes this margin for a limited cool-down window w_{shock} after the sudden-response event in Equation no. (30).

$$\kappa_N(t) = \kappa_N + \Delta\kappa_{shock} \cdot \mathbb{1}[\, t \text{ is within } w_{shock} \text{ of the most recent shock event} \,] \tag{36}$$

Here, $\Delta\kappa_{shock} > 0$. During the cool-down window the larger margin makes the display requirement more selective. Outside that window, $\kappa_N(t) = \kappa_N$.

7.3 Relationship With the History Based Task Scheduling Process and the Complete System

The History Based Task Scheduling process in Section 6.8 and the necessity gate answer two separate questions. Local push first produces an ordered list of next words. Equation no. (13) can then exchange only the first and second words under its residue/support condition. The one-hop fallback can fill unoccupied display positions. Necessity gating is executed after these ranking steps and decides whether the resulting ranked row reaches the interface.

This order preserves the difference between ranking and display. The first-ranked word remains the same internal prediction even when the gate keeps the visible row empty. That ranked row is retained for the next transmission-fidelity outcome update.

Algorithm 1 Updated

The complete system order for one committed word and the following prediction is denoted below.

1. Each raw keypress updates the cadence tracker and the typing-rate tracker. A backspace also updates the correction count.

2. At a space or Enter boundary, tokenize the current buffer into lowercased alphanumeric/apostrophe word units.
3. Before the new word is added to the sentence, resolve the previous pending prediction outcome for transmission fidelity. Edges carrying the non trivial diffusion mass are used for this update. The selected destination receives outcome 1 and the ranked unselected destinations receive outcome 0.
4. Read the accumulated cadence window and calculate the current valence for the selection event.
5. When a previous word exists in the sentence, update the raw edge coefficient and signed value for the selected connection, update the ordinary suppression accumulator, update the recent active-prey set, apply compounding reinforcement when more than one selected edge remains active, and test the sudden negative response.
6. Record the selected word in the context-support history and append it to the sentence.
7. Add the four-feature observational typing-sub-state sample and close the cadence window.
8. Start the next-word prediction from the last k committed words. The current system uses $k = 3$.
9. Build the fan-dispersion seed and read the current $\alpha_{effective}$ from the keypress-derived typing-rate streak, with the minimum value $\alpha_{min} = 0.02$.
10. Run the signed local push and calculate Shannon entropy from the positive part of the complete diffusion output.
11. Exclude the seed-context words from the returned candidates and rank the remaining positive words by diffusion value.
12. Calculate H_{norm} using the number of positive diffusion-output nodes.
13. When two ranked words are available, apply the Section 6.8 top-two residue/support condition. At most the first and second words are exchanged.
14. Keep at most three ranked words. When fewer than three are available, fill remaining positions when possible from unsuppressed one hop outgoing associations of the most recent word. Those fallback edges do not forward diffusion.
15. Store this ranked row as the pending prediction whose outcome is resolved when the human commits the next word.
16. Calculate the necessity value for the first-ranked word from H_{norm}, the current runtime urgency term, and the greatest exhaustion value across the recent seed context. In the immediate word-boundary sequence, the cadence window has already closed, so this urgency input is neutral.
17. Apply the ordinary or shock-adjusted necessity margin. A necessity value above the margin displays the ranked row. Otherwise, the row remains internal and the visible suggestion row is empty.
18. After T_{grace} has elapsed, the next engine event can run the settling process described in Section 6.9.5, adjusting stored unconfirmed compounding deltas, clearing active-prey bookkeeping, and resetting the typing-rate streak.

---

8. Privacy Related Processes and Disadvantages

8.1 Processing the System Data Inside a Single Device

The main advantage of this architecture in terms of privacy comes from processing the system data inside a single device. The graph and the earlier ordinary typing speed value used for Δt_{ref} are processed in that device. (refer Section 6.2) The update processes in Section 3, Section 4, Section 6 and Section 7 are also processed inside the device.

The core prediction, graph update, timing, and gating processes do not require an external inference server. In a local only deployment, the keypress derived values and graph can remain on that device. The web playground used in Section 10 adds an optional remote state saving layer, so a deployment can choose between a local only state and a synchronized state.

This privacy advantage is limited to the data being processed inside one device. It is different from the mathematical process of differential privacy. (refer [6]) Differential privacy uses separate calculations to limit what can be found from stored information. The process described in this section does not give that similar result when another human reads or copies the data stored in that device.

If the IntentSpider system shares the graph state between several devices through a cloud server, then the data does not remain inside a single device. In that state, the system requires a separate privacy process for the data transmitted between those devices.

8.2 The Differential Privacy Process and More Research

The privacy process in this version is based on local processing, protected storage and direct control over the saved graph. Differential privacy is a separate mathematical process. (refer [6]) Applying that process to a signed and function typed graph creates two separate areas for more research. The first area is adding a calibrated noise value to a graph which has a non Euclidean structure. That noise value requires its own mathematical process for calculating the privacy result.

The second area is related to the information accumulated in the graph across a longer period. One stored data object and a general personal characteristic are different types of information. Which means that, limiting what can be found from one stored data object does not directly limit what can be found from the accumulated graph structure. Information such as the gender of the human and another personal characteristic should not be identified and used as a system feature from this graph.

Due to that reason, the privacy process used here keeps the graph processing local when the local deployment is used, protects the stored state and gives the human a direct process for inspecting and deleting the personal graph. Differential privacy for this signed and function typed graph remains a separate process which requires its own graph based calibration.

8.3 Identification Through the Typing Interval Pattern Values

Section 6.2 changes the word list based valence scoring process into the typing interval pattern based process. From this process, it does not need to read the content of a word for calculating val'(c_t). This gives a one advantage in privacy. But it also adds a separate disadvantage.

keypress dynamics research uses the time between the keypresses and the correction rate for identifying and safely proceeding with auth. (refer [27]) The Δt_{avg}, Δt_{ref} and κ values used by IntentSpider include the similar type of typing information. These values can be processed inside the device without a remote inference request. A synchronized deployment stores those timing values inside the same protection boundary as the graph.

But if another human reads or copies the stored data of that device, these values can show the specific way how that human types. The graph shows what that human connects

with the other words. The typing interval pattern values show how that human uses the keyboard. Specifically, today with Artificial Intelligence and advanced reverse engineering tools, this is a real possibility. And these are two separate types of personal information.

The Δt_{ref} value and the other per user timing values need the similar stored data protection used for the graph. These values describe the typing pattern of that human. Due to that reason, these values should be treated as sensitive personal typing information together with the graph.

8.4 Protection and Safe Storage of IntentSpider Graph Data

The graph described in Section 3, Section 6 and Section 7 includes the private word connections of a human. From these word connections, information related to health conditions, relationships and the situational vulnerabilities can be identified. This is depending on what that human types and the specific time when those words are typed.

The graph data requires protection while it is stored. This includes an encrypted data storage process. And also, the cloud storage should not be enabled as the default process. The IntentSpider system should allocate the human a direct access setting for inspecting and deleting selected graph parts. This protection is required even in a local only deployment, and it becomes more important when a particular deployment enables graph synchronization or remote persistence.

8.5 About Typing Rate Values and Typing Sub States

The μ_{user} and σ_{user} values in Section 6.9.1 are calculated from the earlier typing rate of that person. These two values are another set of typing based personal information. In a local deployment they can remain inside that device. In the Webnet playground, the optional state saving process can save the rolling timing values together with the graph state. If that stored state is obtained by another personnel (examples include data theft, legal asset freezing, malware), those values can become available together with Δt_{ref} and the graph. Due to that reason, the μ_{user} and σ_{user} values need the similar stored data protection.

In here, the TRD(t) is a variable which does not have a positive or a negative direction. (TRD(t) is a scalar quantity) The discovered per user typing sub states in the subsection 6.9.6 include several typing patterns instead of a one value.

The current IntentSpider process uses the four value grouping described in Section 6.9.6 for finding repeating typing sub states. These groups can include more information about the typing behavior than one timing value.

9. The Processing and Storage Amounts Required Inside a Single Device

Due to the variances in modern Operating Systems in Tablet, Mobile and Web Interfaces, this section outputs the approximate calculation values for the storage and the processing amounts. They are calculations made under the assumptions denoted below. These calculations provide information to understand whether the system is possible inside a mobile device into a some extent.

The calculations in this section are theoretical estimate values. Section 10 gives the measured processing time, browser processor usage, saved graph amount, WebAssembly memory and JavaScript memory values from the experiment. Due to that reason, the calculated values in this section can be compared with the measured values in Section 10.

9.1 Storage Requirements

Suppose a human who types heavily builds approx. $|V| \approx 5 \times 10^4$ number of different words within the timeframe of several years. And suppose the average node degree is $d \approx 20$. The earlier calculation used d across the two combined channels. But Section 3.2 now uses a one single edge set. So value d now outputs the degree for each node inside that one edge set.

For a simple minimum calculation, suppose one edge uses four 4 byte values and approx. 8 bytes for the adjacency or index information. In that state, one edge uses approx. 24 bytes and gives the following calculation.

$|V| \times d \times 24$ bytes $\approx 5{\times}10^4 \times 20 \times 24 \approx 24$ MB.

The current IntentSpider edge includes more values than this minimum calculation. It stores the target, the raw coefficient, transmission fidelity, the signed value, the negative accumulator, suppression and shock state and several time values. Due to that reason, the 24 MB value above is a minimum structural estimate and not a byte count of the current C++ edge object. The saved graph measured in Section 10 occupied approximately 2.209 MiB for the state used by the main experiment.

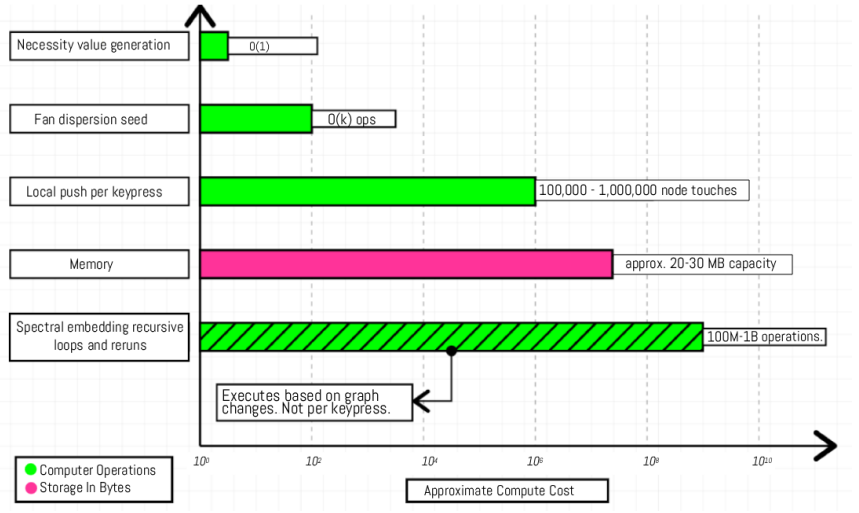

Figure 14. Approx. storage and processing cost of the IntentSpider system based on assumptions (refer section no. (9))

The typing-interval based valence signal in Section 6.2 replaces the earlier word list. That word list was a old process with a few thousand entries. And it used an amount lower than 1

MB. The new process keeps several values for each human. This includes Δt_{ref} and a short rolling window of the recent Δt values and the backspace counts. Which means that, this amount is small even when compared to the word list it replaces.

The two amounts are small when compared with the memory and storage capacities reported for current smartphones. Counterpoint Research reported that the global average smartphone DRAM capacity reached 8.4 GB in December 2025 (refer [34]). Omdia separately reported Q1 2026 global averages of 8.3 GB DRAM and 279 GB NAND storage (refer [35]). These market values are used only as scale references; they are not measurements of the IntentSpider deployment itself. This version of IntentSpider also did not separately measure allocator overhead, garbage collection, operating-system caching, or storage-management costs.

Section 6.9 adds several more values for each human. This includes μ_{user} and σ_{user} for the typing rate. And it includes $\mu_{val,user}$ and $\sigma_{val,user}$ for the valence value. In addition to that, it includes the current sustained period length ℓ(t) and a shock time value. Each edge included in the shock process also keeps a shock state and a decay clock (refer Section 6.9.4). These values use an amount within tens of bytes together. Meaning, their amount is small in the similar way as Δt_{ref}.

The discovered typing sub state groups in Section 6.9.6 need a separate note. Equation (33) gives five values and the current grouping process uses four of those runtime values. It groups them using the nearest existing center and a distance margin value, with a maximum number of groups. These group center values use a small storage amount when compared to the graph. And this grouping does not change the next word ranking.

9.2 The Processing Time for Keyboard Input and Next Word Prediction

The local push process in Section 4.3 creates the main graph processing amount for a next word prediction. Raw character keypresses update the typing time values. The graph observation and the local push prediction are executed when the current word is committed. In a graph with coefficients which are positive values and a fixed edge set, the original local push result has the $O(1/(\alpha\epsilon))$ node touching limit. With $\alpha \approx 0.15$ and $\epsilon \approx 10^{-5}$, this gives an amount around 10^5 to 10^6 touches. The current IntentSpider process uses $\epsilon = 10^{-4}$ and a maximum of 200,000 push operations for one local push call. Section 10 gives the measured value from the playground. The median processing time for the complete word boundary prediction handler was 2.40 ms.

Building the fan dispersion starting point from Equation (19) uses O(k) work for k recent words. It also requires one dispersion value lookup for each word. This amount is small when it is compared to the local push process.

The necessity value from Equation (15) in Section 7.3 adds another small amount. This process outputs a fixed coefficient based addition of three values which are already calculated for each next word. Which means that, it uses time complexity Big O(1) processing for each next word. So, due to that reason, there will be no changes to the processing order denoted above.

Two processes from Section 6.9 add another processing amount at selection events. The shared cluster reinforcement in Equation (26) uses an amount which changes with the number of recently selected edges in P(t). The sudden negative response uses a small fixed amount from the rolling values. In here, P(t) includes the recently selected edges. These two processes add processing operations to the complete system.

The growing radius process in Section 6.9.3 is different. The O$(1/(\alpha\epsilon))$ limit is no longer calculated with a fixed α value. The earlier 10^5 to 10^6 node touch calculation uses $\alpha \approx 0.15$. Once $\alpha_{effective}$(t) reduces during a sustained typing rate difference period, that earlier

calculation becomes a lower processing amount and not the ordinary processing amount for that period.

In the unrestricted equation, moving $\alpha_{effective}$ closer to 0 increases the processing amount from that same limit. The current IntentSpider process keeps $\alpha_{effective}$ at or above $\alpha_{min} = 0.02$ and also keeps the maximum local push operation amount. Which means that, the growing radius process has a minimum α value in the system used by Section 10.

The working value is $\alpha_{min} = 0.02$. This still allows the diffusion radius to increase during a sustained hexarousal period while keeping a minimum value for $\alpha_{effective}$.

9.3 Comparing Against Already Existing Processes

This below table (2) gives a feature based comparison between IntentSpider Webnet and the related research systems discussed in Section 2. The documented architectures in those references do not define a display process corresponding to IntentSpider Equation (15), so the table compares the mechanisms which are described directly in the cited work.

Table no. 2. A comparison between the already existing external processes and the IntentSpider Webnet.

Computer Arch.	Inside a device or without an external server?	Supports the starting state without earlier data?	Supports valence?
classic n gram architecture (historical predictive-text background) (refer [14,17])	Yes	No	No
Federated RNN keyboard (refer [10])	Local device training/inference with server aggregation during federated learning	Starts from an existing shared model	No
DualNet, CLS ER, and ITDMS (refer [21,2,22])	Not Applicable	Not Applicable	No
IntentSpider Webnet V1	The main prediction and update processes can run locally	The graph can begin from an empty state and increases with the entered word connections	Yes (mentioned in main section 6.)

9.4 The Processing Amount of the Periodic Graph Embedding Recalculation

Calculating all exact eigenvectors directly from a $|V| \times |V|$ Laplacian is not possible for the graph size assumed in the storage calculation. In that calculation, $|V| \approx 5 \times 10^4$. A more direct and applied simplifying can use $O(|V|^3)$ processing. Which means it uses an amount approx. 100,000,000,000,000 operations (more than 100 Trillion Ops per calculation). This amount is outside a processing amount available inside a general mobile device such as an iPhone.

But Equation (37) needs only the two leading eigenvectors. Iterative processes such as the Lanczos iteration can calculate these two values from the sparse structure of the graph. Another option is the power iteration process on a shifted or inverted operator. The approximate processing amount is $O(k \cdot nnz(L) \cdot \text{number of iterations})$. In here, $k = 2$. And $nnz(L) \approx |V| \times d \approx 5{\times}10^4 \times 20 \approx 10^6$ entries with values which are not 0. The number of iterations for the first two eigenvectors can be from tens to a lower number of hundreds. This is depending on the difference between λ_2 and λ_3.

These assumptions output approx. 10^8 to 10^9 basic operations for one iterative estimate. This amount is greater than one local push call. In the current developer process, IntentSpider creates the undirected form of the transmission graph, uses its largest connected component and calculates the two directions through a repeated power iteration process. The WebAssembly prediction process measured in Section 10 runs separately from this graph embedding process. Due to that reason, the measured next word prediction time in Section 10 does not include a recalculation of the two dimensional embedding.

10. Experiments and Validation

10.1 The Evaluation Data and the Next Word Events

The evaluation data was fixed before the keyboard results were scored. The source is the conversational data set preserved inside reference [36]. The text was kept in its original order. Alphabetic words, apostrophe words and numerical values were considered as individual word units. Lines containing the project redaction markers and lines containing emoji characters were not used. A source line also required at least two word units before it could produce a next word event.

The complete fixed evaluation data contains 1,000 next word events, named EV0001 through EV1000. The main keyboard experiment uses EV0001 through EV0500. The first event contains the context "What" and the correct next word "is". The second event contains "What is" and the correct next word "your". The events continue in the original conversation order. Which means that, the experiment keeps the turning and changing structure of the conversation instead of converting it into a shuffled word list.

The 1,000 events come from 65 contributing turns with 1,074 word units and 422 different words. 244 of those different words, or 57.8%, appear one time. 236 of the 1,000 next word events have a correct next word which appears one time. 92.3% of the different two word previous contexts appear one time and 98.5% of the different three word previous contexts appear one time. In addition to that, 157 of the 166 repeated one word contexts are followed by more than one different next word. The largest observed number of different next words after one repeated previous word is 30. These values describe the highly changing conversational text used by the experiment. The complete word/context measurements are published with the other output files in reference [45].

10.2 The IntentSpider State and the Evaluation Process

The IntentSpider starting state was prepared before the 500 event experiment. The exact saved state information used for the experiment is included with the other output files in reference [45].

For IntentSpider, every evaluation event was entered through the ordinary playground input process. The three visible next words were recorded before the correct next word was entered. The correct word was then entered and the ordinary IntentSpider update continued before the next event. Suggestions were not selected during this collection. The complete 500 event IntentSpider result is provided in reference [40].

10.3 Collection From the Existing Keyboard Systems

The same fixed events were used with Google Gboard, Samsung Keyboard, Microsoft SwiftKey and OpenBoard. The previous context was entered into each keyboard and the first three visible suggestions were recorded in their displayed order. The correct next word was entered after those values were recorded. If fewer than three suggestions were displayed, the remaining positions were left empty. A record which could not be read safely was marked REVIEW and was not used inside the scored denominator. A visible state with no suggestions remained as a valid no suggestion event. The individual result files are Samsung Keyboard [41], Google Gboard [42], Microsoft SwiftKey [43], and OpenBoard [44]. The IntentSpider result file is reference [40]. The paired comparison and supporting calculations are provided with the other output files in reference [45].

Gboard was intentionally evaluated with its cloud connected personalization and synchronization functions enabled. Due to that reason, the four keyboard set includes a cloud connected commercial keyboard condition alongside the other systems. The final checked

Table 5. Keyboard and device conditions used for the next word collection.

System	Device	Operating system	Prediction condition
Samsung Keyboard	Samsung Galaxy A01 (SM-A015G/DS)	Android 10	Samsung Keyboard 3.5.45.2, predictive text on, auto replace and spell correction disabled
Google Gboard	Google Pixel 4 XL	Android 10	suggestions on, autocorrection off, cloud connected personalization and synchronization enabled
Microsoft SwiftKey	Google Pixel 4 XL	Android 10	version 9.13.14.5, prediction on, autocorrect off, emoji predictions off
OpenBoard	Google Pixel 4 XL	Android 10	open source AOSP and LatinIME derived keyboard, prediction enabled

result includes 444 Samsung Keyboard events, 187 Gboard events, 454 Microsoft SwiftKey events and 435 OpenBoard events.

10.4 Scoring the Next Word Predictions

IntentSpider returns three next words. Due to that reason, the first three displayed positions were used for all five systems. First rank accuracy counts an event as correct only when the first displayed word equals the correct next word. Top 3 accuracy counts the event as correct when the correct next word appears in any of the first three positions.

MRR@3 gives a value of 1 when the correct word is in position 1, 1/2 in position 2 and 1/3 in position 3 before taking the average. Suggestions displayed is the percentage of valid events where at least one suggestion was visible. The scoring used exact target matching. REVIEW and unresolved MISSING records remained outside the denominator of that keyboard. An explicit no suggestion state remained as a valid scored event.

10.5 The Main 500 Word IntentSpider Result

Across the complete 500 event IntentSpider result, 72 correct next words appeared in the first position. Which gives 14.40% first rank accuracy. The correct next word appeared inside the first three positions in 139 events, giving 27.80% Top 3 accuracy. MRR@3 was 0.2003. IntentSpider displayed at least one suggestion in 444 of the 500 events, or 88.80%. The full 500 event IntentSpider result is provided in reference [40]. The paired comparison and statistical support files are provided in reference [45].

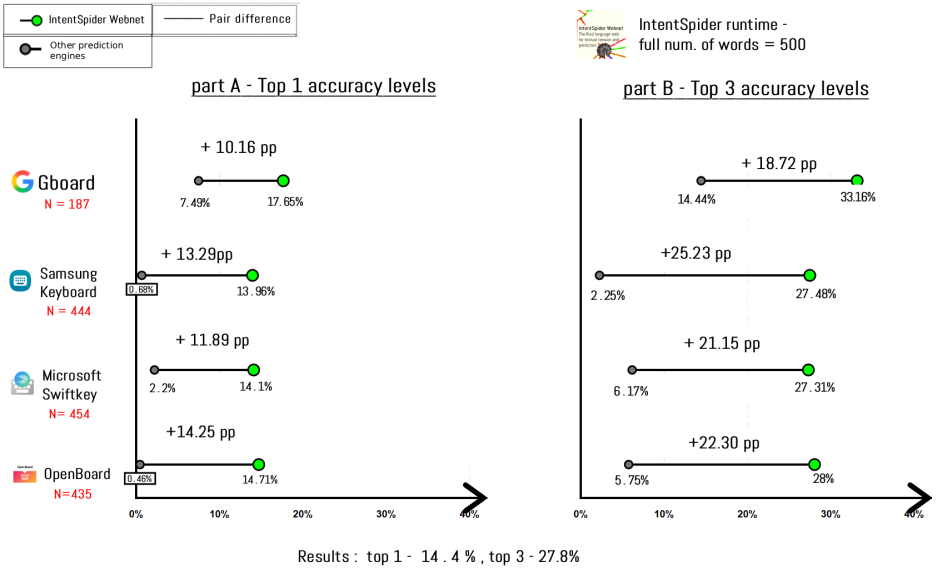

Figure 15. First rank and Top 3 next word accuracy for IntentSpider and the four other keyboards. The complete 500 event IntentSpider result is also included.

10.6 Comparison With the Four Existing Keyboards

Figure 15 compares IntentSpider with each keyboard using the events which have a valid result for both systems. The exact paired comparison values and statistical support are provided in reference [45].

Among the four existing keyboard systems, Gboard produced the greatest prediction accuracy in this evaluation. In 187 events, Gboard reached 7.49% first rank accuracy and 14.44% Top 3 accuracy. IntentSpider reached 17.65% first rank accuracy and 33.16% Top 3 accuracy in those same events. The differences are +10.16 and +18.72 percentage points.

In the 444 Samsung Keyboard events, IntentSpider reached 13.96% first rank and 27.48% Top 3 accuracy. Samsung Keyboard reached 0.68% and 2.25%.

In the 454 Microsoft SwiftKey events, IntentSpider reached 14.10% first rank and 27.31% Top 3 accuracy. Microsoft SwiftKey reached 2.20% and 6.17%.

In the 435 OpenBoard events, IntentSpider reached 14.71% first rank and 28.05% Top 3 accuracy. OpenBoard reached 0.46% and 5.75%.

Table 6. Exact next word accuracy from the events available for both IntentSpider and each keyboard.

Keyboard	N	IntentSpider first	Keyboard first	IntentSpider Top 3	Keyboard Top 3
Gboard	187	17.65%	7.49%	33.16%	14.44%
Samsung Keyboard	444	13.96%	0.68%	27.48%	2.25%
Microsoft SwiftKey	454	14.10%	2.20%	27.31%	6.17%
OpenBoard	435	14.71%	0.46%	28.05%	5.75%

10.7 Difference in Prediction Accuracy and Statistical Range

Several prediction events can come from the same source message. Due to that reason, the 95% ranges in Figure 16 were calculated from 10,000 repeated samples of the source messages while keeping the events from each sampled message together.

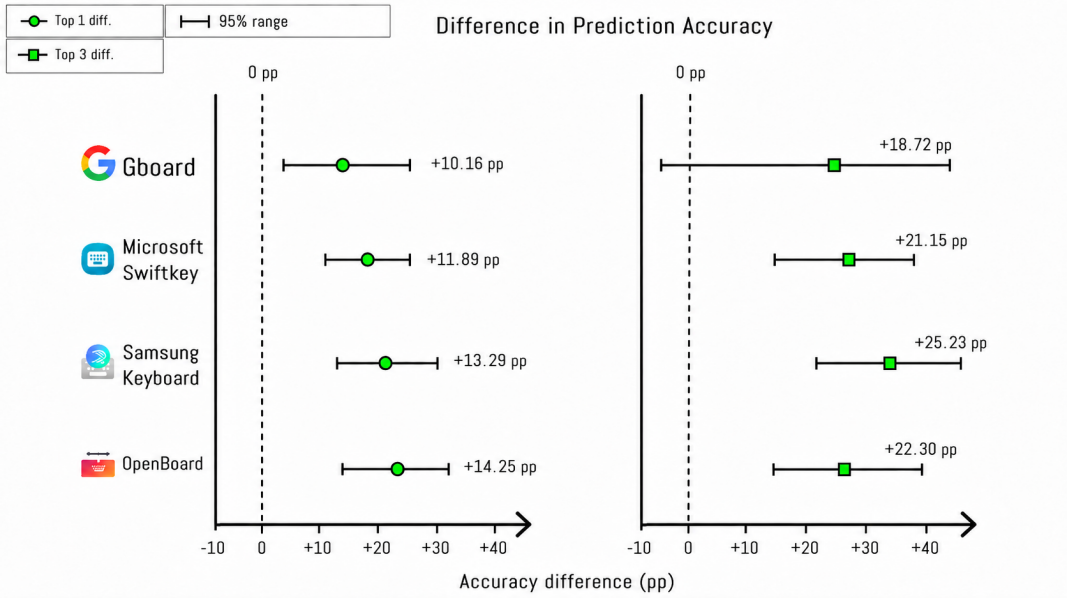

Figure 16. Accuracy differences between IntentSpider Webnet and other keyboard systems in use. Positive values represent greater measured accuracy for IntentSpider. The horizontal lines show the 95% range from 10000 sample selections (repeated) of the original dataset containing short form text conversations. .

For Gboard, the first rank difference is +10.16 percentage points and its 95% range is from -3.3 to +18.9 points. The Top 3 difference is +18.72 points and its range is from -2.6 to +31.1 points. These 187 events come from 13 source message groups.

For Samsung Keyboard, the first rank difference is +13.29 points with a range from +9.3 to +20.3 points. The Top 3 difference is +25.23 points with a range from +20.6 to +32.0 points.

For Microsoft SwiftKey, the first rank difference is +11.89 points with a range from +9.1 to +15.6 points. The Top 3 difference is +21.15 points with a range from +15.5 to +26.2 points.

For OpenBoard, the first rank difference is +14.25 points with a range from +10.0 to +21.2 points. The Top 3 difference is +22.30 points with a range from +16.8 to +30.9 points.

In addition to that, the same event pairs were calculated using the exact McNemar test and Holm correction across the eight first rank and Top 3 comparisons. All eight Holm adjusted values were below 0.05. The full calculations are included in reference [45].

10.8 UI Interfaces Used During the Collection Process

Figure 17 shows the IntentSpider playground and the four keyboard interfaces used during the collection. These images show the interfaces and the visible predictions by those keyboards (usually 3) were recorded.

Table 7. Prediction accuracy differences, source message 95% ranges and the Holm corrected exact paired results.

Keyboard	First rank difference [95% range]	Top 3 difference [95% range]	Holm exact p, first / Top 3
Gboard	+10.16 [-3.3,+18.9] pp	+18.72 [-2.6,+31.1] pp	0.00256 / 0.0000153
Samsung Keyboard	+13.29 [+9.3,+20.3] pp	+25.23 [+20.6,+32.0] pp	1.75×10^{-15} / 1.69×10^{-26}
Microsoft SwiftKey	+11.89 [+9.1,+15.6] pp	+21.15 [+15.5,+26.2] pp	5.48×10^{-11} / 7.46×10^{-16}
OpenBoard	+14.25 [+10.0,+21.2] pp	+22.30 [+16.8,+30.9] pp	3.60×10^{-16} / 2.55×10^{-18}

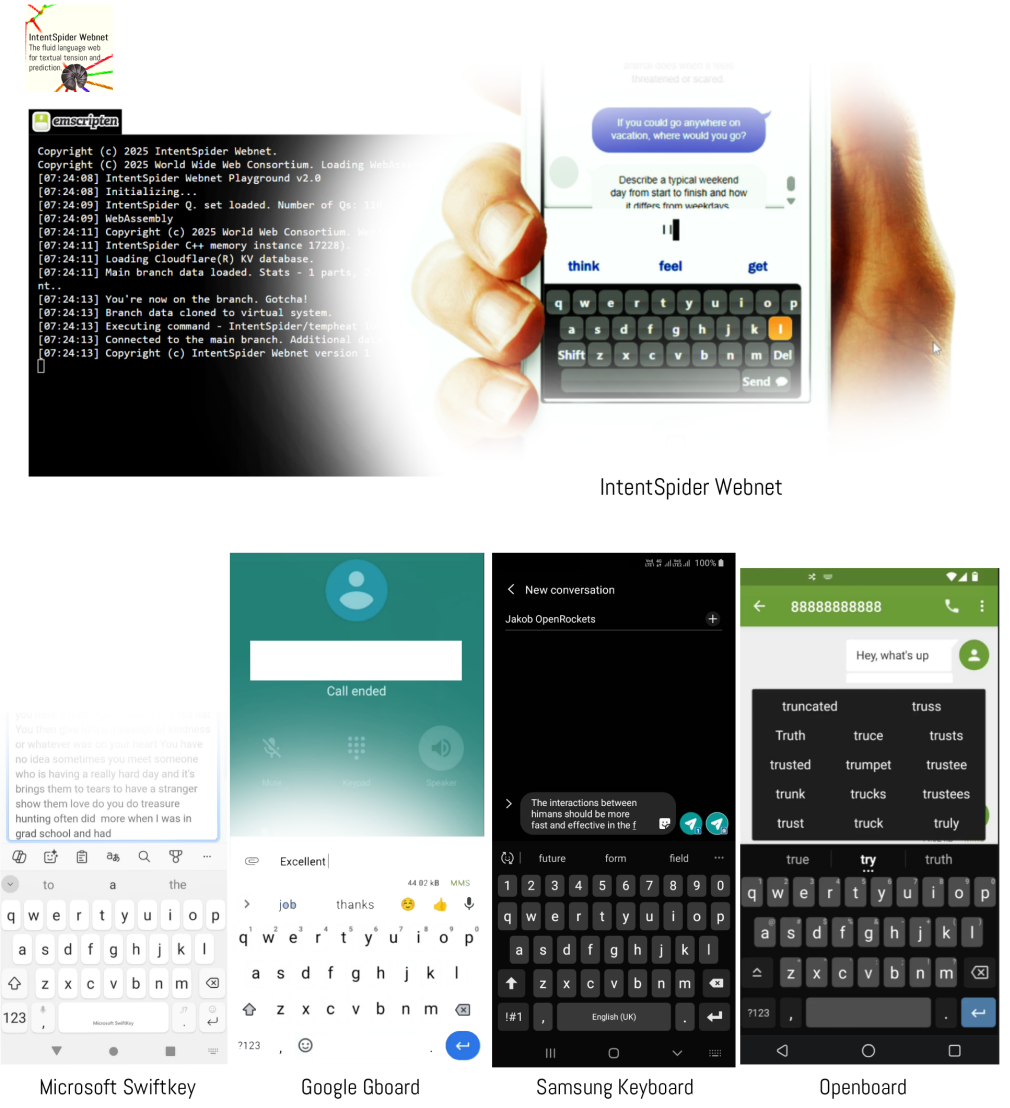

Figure 17. Interfaces used during the evaluation and validation section. The first image is the IntentSpider Webnet web research preview followed with Microsoft SwiftKey, Google Gboard, Samsung Keyboard, and OpenBoard.

10.9 Prediction Accuracy Under Different Word and Previous Context Conditions

The main 500 event result can also be separated by properties of the evaluation text. Among the first 500 events, 117 correct next words are words which appear only one time inside the complete 1,000 event data. IntentSpider reached 4.27% first rank and 9.40% Top 3 accuracy in this group. In the other 383 events, where the correct next word appears more than one time, IntentSpider reached 17.49% first rank and 33.42% Top 3 accuracy. Suggestions were displayed in 88.89% and 88.77% of those two groups.

A second grouping uses the immediately previous word. In 378 events, the same previous word appears elsewhere in the evaluation data and is followed by several different next words. IntentSpider reached 10.85% first rank and 21.69% Top 3 accuracy in this group. In the other 122 previous word cases, the result was 25.41% first rank and 46.72% Top 3. The calculations are provided in reference [45].

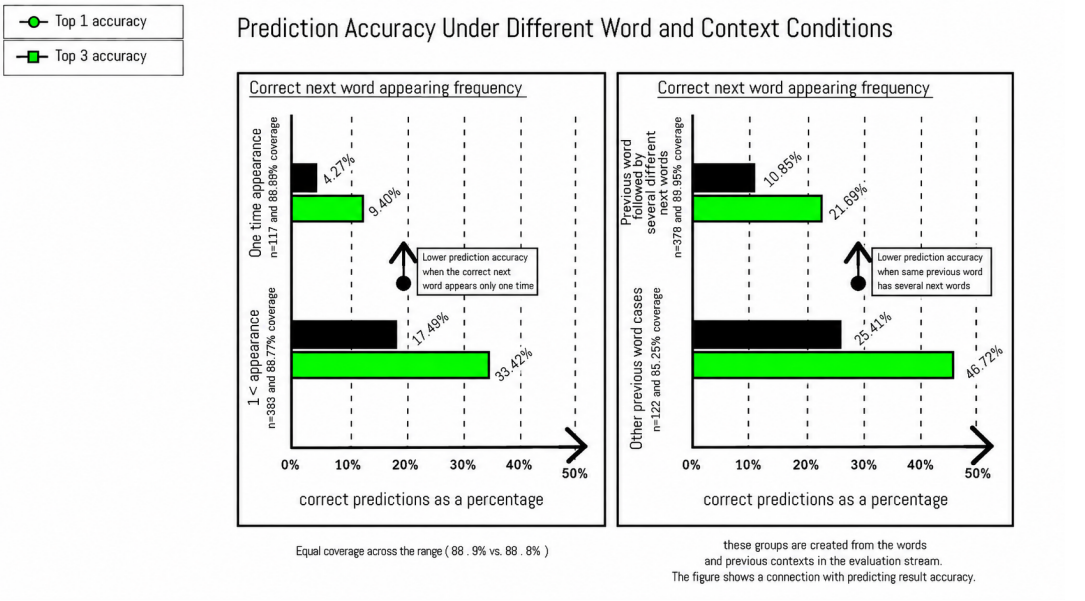

Figure 18. IntentSpider prediction accuracy under two word and previous context conditions in the 500 event evaluation.

10.10 Position of the Next Word Inside the Sentence

Figure 19 shows the position of the predicted word inside the sentence and the accuracy changes across those positions.

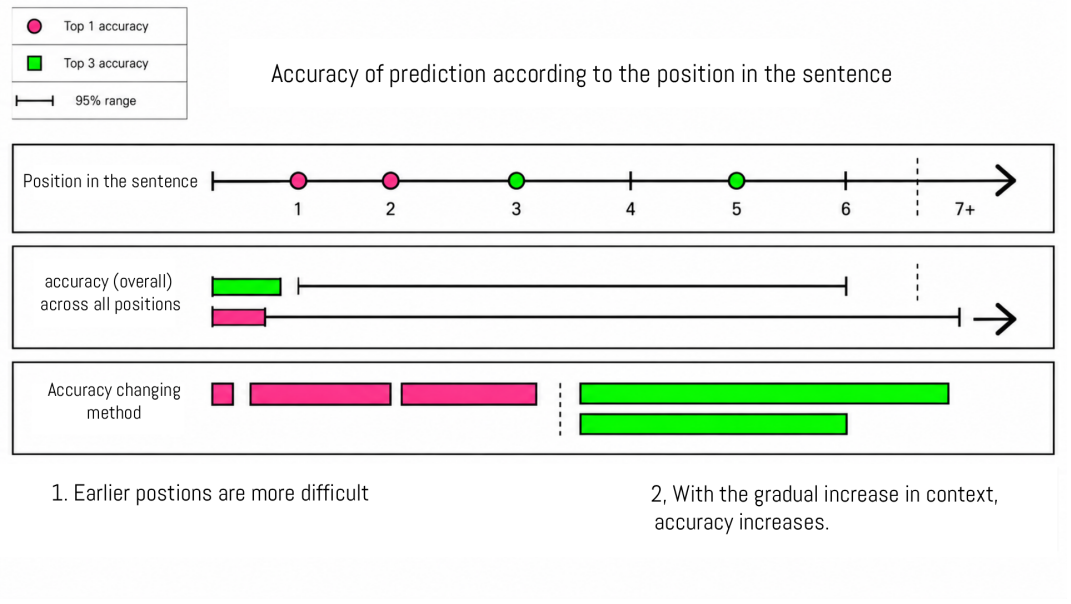

Figure 19. Prediction accuracy according to the position of the next word in the sentence.

10.11 Measured Processing Time, Memory and Saved Data Amounts

The production playground was measured during a 90 second human typing behavior workload. Fifty word boundary prediction events were recorded. The median processing time was 2.40 ms. The 95th percentile was 3.66 ms, the 99th percentile was 8.08 ms and the maximum was 11.8 ms. All 50 measured prediction events completed below 16.67 ms. The maximum value happened on the first prediction event. After that first event, the 95th percentile was 3.56 ms and the maximum was 4.2 ms.

This measurement includes the JavaScript keyboard event process, the WebAssembly prediction call, JSON handling, the suggestion display update, the debugging value update and the text display update. The runtime and latency outputs are published in the supporting output directory in reference [45].

A second measurement used a 30 minute human typing behavior workload in a dedicated Microsoft Edge browser environment on Windows 11 Pro with an Intel Core i7 1165G7 processor. The complete dedicated Edge process tree used a mean 0.281% of the total machine CPU. Its 95th percentile was 1.342%, its 99th percentile was 2.046% and the maximum was 2.458%.

The saved data and memory values were measured separately. The complete Edge process tree had a mean working memory of 788.4 MB. The loaded WebAssembly linear memory region was 64.0 MiB. The used JavaScript heap snapshots were 36.6 and 40.6 MiB. The saved graph occupied approximately 2.209 MiB. The saved temporary IntentSpider state occupied 2.151 MiB and the WebAssembly program file occupied 0.205 MiB. These values are provided in reference [45].

10.12 Controlled Changes Inside IntentSpider

For these measurements, the same fixed IntentSpider state was loaded before each change and the same 500 events were used in every condition. A new engine instance was used for each condition. Suggestions were not selected. The complete result is provided in reference [45].

Table 8. Measured processing, saved data and memory values.

Measurement	Value	Measurement level
Prediction processing median	2.40 ms	production word boundary handler, n=50
Prediction processing p95 / p99 / max	3.66 / 8.08 / 11.8 ms	same 90 second workload
Dedicated Edge CPU mean	0.281% of machine	complete dedicated browser process tree
Dedicated Edge CPU p95 / max	1.342% / 2.458%	complete dedicated browser process tree
WebAssembly linear memory	64.0 MiB	loaded WebAssembly memory region
JavaScript heap used	36.6 to 40.6 MiB	page level used JavaScript heap
Saved graph	2.209 MiB	serialized graph file
Saved temporary state	2.151 MiB	serialized temporary state file
WebAssembly program	0.205 MiB	stored program file
Complete Edge working memory mean	788.4 MB	complete browser process tree

Table 9. Controlled changes inside IntentSpider using the same 500 next word events.

Condition	First rank	Top 3	Suggestions displayed	Change made
Full IntentSpider state	8.4%	12.8%	81.0%	no process removed
Without loaded temporary state	8.2%	12.8%	81.0%	use the saved graph without loading the temporary state
Fixed keypress timing	8.8%	13.2%	81.8%	keep the timing interval fixed
Without runtime valence update	8.6%	13.2%	80.2%	stop the runtime valence contribution
Fixed diffusion radius	8.4%	12.8%	81.0%	keep the diffusion restart value fixed
One word seed	31.4%	42.6%	93.4%	use only the most recent word in the seed
Without compounding reinforcement	8.6%	12.8%	80.2%	stop the shared reinforcement addition
Without residue and support choosing	8.6%	12.8%	80.8%	disable the optional first and second word choosing rule
Without necessity gating	16.8%	24.6%	99.8%	return the ranked words without the display requirement
Without signed suppression channel	7.8%	11.2%	55.8%	remove the signed suppression channel

Using only the most recent word in the seed increased the first rank accuracy from 8.4% to 31.4% and the Top 3 accuracy from 12.8% to 42.6%. Which means that, the number of recent words used by the seed produced one of the largest changes in this experiment.

Removing the necessity gate increased first rank accuracy to 16.8%, Top 3 accuracy to 24.6% and suggestions displayed to 99.8%. In the full condition, the gate withheld suggestions on 94 events. When those ranked words were returned in the condition without the gate, 42 first position predictions and 59 Top 3 sets contained the correct next word. This shows the accuracy and display difference created by the current necessity gating values in this fixed data.

Removing the loaded temporary state changed one of the 500 visible prediction rows. 499 visible rows remained the same as the full condition. Top 3 accuracy remained 12.8% and suggestions displayed remained 81.0%. Which means that, the saved graph carried almost all of the visible next word behavior measured in this particular 500 event run.

Removing the signed and suppression channel reduced suggestions displayed from 81.0% to 55.8%. First rank accuracy changed from 8.4% to 7.8%, and Top 3 accuracy changed from 12.8% to 11.2%. The largest change from this condition is the amount of ranked suggestions which reach the interface.

The condition without the loaded temporary state was also aligned to the keyboard event sets. It retained greater first rank and Top 3 point values than Samsung Keyboard, Microsoft SwiftKey and OpenBoard. Against Gboard, the first rank value was equal at 7.49%, while the Top 3 value was 14.97% for IntentSpider and 14.44% for Gboard. These values are included in reference [45].

10.13 Result Files

The complete experiment resource directory is provided in reference [39]. The five 500 word evaluation result files are provided directly in references [40] to [44]. The other output files used by the evaluation and validation section are provided in reference [45]. The conversational data set used to train the Global Person Mode in the Research Preview is reference [37]. The IntentSpider web portal and the Interactive Research Preview are references [46] and [47]. The IntentSpider engine source code is reference [48].

11. Conclusion

The function typed and diffusion based architecture is the main system described in the IntentSpider system for the on device next word prediction feature. In this system, every edge receives a position from a current ongoing transmission fidelity value. This value can increase or reduce in all of those two directions. Which means that, the relevant position of an edge is not decided by a one way consolidation clock, but rather, it is decided based on whether that edge has a reliable ability to predict the next word which the human may select. In addition to that, the initial seed values for the diffusion process are not created using only the single most recent word. Those values are created using a combination of several recent individual words. And the coefficients are based on the scattered distribution of those word connections. (refer Section 3 and Section 4)

The IntentSpider system combines already existing research processes into a different predictive architecture. On device text prediction, local computation, graph diffusion, reinforcement, signed graph methods, typing interval pattern values and spectral graph embedding are already existing research areas into a some extent. IntentSpider gives graph edges a function type from a current ongoing and freely reversible transmission fidelity value, and reads that graph through diffusion with the recent word combination described above. This is also the literal reason for the name IntentSpider: the main object is the human's changing web of word connections, the edges have thread like functional roles, and prediction is read by sending signal through the reliable part of that web. The spider biology is the design analogy which led to the layered signal, residue, and changing thread ideas used by the system.

Section 5 identifies the blindness to the valence in a graph which only strengthens a selected word connection. Section 6 adds the signed and valence gated extension, using typing interval patterns and correction rate rather than a fixed emotional word list. Section 7 then places necessity gating after the ranking process, so the question of which next word is ranked and the question of whether that ranked word is displayed remain separate processes. The controlled experiment in Section 10 shows this distinction directly: removing the necessity gate returned more suggestions and exposed additional exact next word hits on the same replay.

Section 6.9 adds the separate hexarousal axis from the size of the typing rate difference relative to the ordinary typing rate of that human. Around this axis, IntentSpider includes the compounding reinforcement process, the growing diffusion radius, the sudden negative response, the settling process and the discovered typing sub states. Section 4.5 gives the two dimensional spectral embedding of the same graph. This process transforms the word prefix diffusion outputs into a path without changing the prediction process.

The completed evaluation gives the measured result for the IntentSpider system. Across the main 500 next word events, IntentSpider reached 14.40% first ranked accuracy and 27.80% Top 3 accuracy. On each keyboard specific shared event set, IntentSpider produced the greater measured first ranked and Top 3 accuracy than Gboard, Samsung Keyboard, Microsoft SwiftKey and OpenBoard. Among these existing keyboards, Gboard produced the greatest measured accuracy. On the shared Gboard events, IntentSpider reached 17.65% against 7.49% at the first rank and 33.16% against 14.44% inside the Top 3 words.

The experiment also measured the processing behavior. The playground produced a 2.40 ms median prediction time and a 3.66 ms 95th percentile across 50 word boundary predictions. The controlled changes produced the greatest differences when the number of recent words used in the seed was reduced to one and when the necessity gate was removed. Removing the loaded temporary state left 499 of the 500 visible prediction rows unchanged in the same comparison. Which means that, the saved graph already carried almost all of the visible next word ranking behavior in that specific comparison.

The published experiment resources are provided in reference [39]. The five 500 word evaluation result files are provided in references [40] to [44]. The supporting statistical, runtime, controlled change, state, repeatability, word and context output files are provided in reference [45].

References

[1] R. Andersen, F. Chung, and K. Lang. Local graph partitioning using PageRank vectors. In 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS 2006), pages 475–486, 2006.

[2] E. Arani, F. Sarfraz, and B. Zonooz. Learning fast, learning slow: A general continual learning method based on complementary learning system. In International Conference on Learning Representations (ICLR), 2022.

[3] D. Cartwright and F. Harary. Structural balance: A generalization of Heider's theory. Psychological Review, 63(5):277–293, 1956.

[4] F. Chung. The heat kernel as the PageRank of a graph. Proceedings of the National Academy of Sciences, 104(50):19735–19740, 2007.

[5] M. Dorigo. Optimization, Learning and Natural Algorithms. PhD thesis, Politecnico di Milano, 1992.

[6] C. Dwork. Differential privacy. In 33rd International Colloquium on Automata, Languages and Programming (ICALP), pages 1–12, 2006.

[7] C. Genest, S. Weerahandi, and J. V. Zidek. Aggregating opinions through logarithmic pooling. Theory and Decision, 17(1):61–70, 1984.

[8] C. Genest and J. V. Zidek. Combining probability distributions: A critique and an annotated bibliography. Statistical Science, 1(1):114–135, 1986.

[9] F. Glover. Future paths for integer programming and links to artificial intelligence. Computers & Operations Research, 13(5):533–549, 1986.

[10] A. Hard, K. Rao, R. Mathews, S. Ramaswamy, F. Beaufays, S. Augenstein, H. Eichner, C. Kiddon, and D. Ramage. Federated learning for mobile keyboard prediction. arXiv preprint arXiv:1811.03604, 2018.

[11] G. E. Hinton and D. C. Plaut. Using fast weights to deblur old memories. In Proceedings of the Ninth Annual Conference of the Cognitive Science Society, pages 177–186. Erlbaum, 1987.

[12] C. J. Hutto and E. Gilbert. VADER: A parsimonious rule-based model for sentiment analysis of social media text. Proceedings of the International AAAI Conference on Web and Social Media, 8(1):216–225, 2014. DOI.

[13] G. Jeh and J. Widom. Scaling personalized web search. In Proceedings of the 12th International Conference on World Wide Web (WWW), pages 271–279, 2003.

[14] S. Katz. Estimation of probabilities from sparse data for the language model component of a speech recognizer. IEEE Transactions on Acoustics, Speech, and Signal Processing, 35(3):400–401, 1987.

[15] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. Proceedings of the National Academy of Sciences, 114(13):3521–3526, 2017.

[16] J. Gasteiger, A. Bojchevski, and S. Günnemann. Predict then propagate: Graph neural networks meet personalized PageRank. In International Conference on Learning Representations (ICLR), 2019. OpenReview H1gL-2A9Ym.

[17] R. Kneser and H. Ney. Improved backing-off for M-gram language modeling. In IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP), volume 1, pages 181–184, 1995.

[18] J. Kunegis, S. Schmidt, A. Lommatzsch, J. Lerner, E. W. De Luca, and S. Albayrak. Spectral analysis of signed graphs for clustering, prediction and visualization. In Proceedings of the 2010 SIAM International Conference on Data Mining (SDM), pages 559–570, 2010.

[19] J. L. McClelland, B. L. McNaughton, and R. C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. Psychological Review, 102(3):419–457, 1995.

[20] L. Page, S. Brin, R. Motwani, and T. Winograd. The PageRank citation ranking: Bringing order to the web. Technical report, Stanford InfoLab, 1998.

[21] Q. Pham, C. Liu, and S. Hoi. DualNet: Continual learning, fast and slow. In Advances in Neural Information Processing Systems (NeurIPS), volume 34, 2021.

[22] R. Wu, K. Huang, H. Zhang, Q. Liu, J. Guo, J. Deng, and F. Ye. Information-Theoretic Dual Memory System for continual learning. Knowledge-Based Systems, 325:113913, 2025. DOI.

[23] R. Eberle. SCAMPER: Games for Imagination Development. D.O.K. Publishers, 1971.

[24] A. F. Osborn. Applied Imagination: Principles and Procedures of Creative Problem-Solving. Charles Scribner's Sons, 1953 (3rd rev. ed., 1963).

[25] R. F. Foelix. Biology of Spiders, 3rd edition. Oxford University Press, 2011.

[26] R. Hergenröder and F. G. Barth. The release of attack and escape behavior by vibratory stimuli in a wandering spider (Cupiennius salei Keys.). Journal of Comparative Physiology A, 152:347–358, 1983.

[27] F. Monrose and A. D. Rubin. Keystroke dynamics as a biometric for authentication. Future Generation Computer Systems, 16(4):351–359, 2000.

[28] J. A. Russell. A circumplex model of affect. Journal of Personality and Social Psychology, 39(6):1161–1178, 1980.

[29] J. B. Walther. Computer-mediated communication: Impersonal, interpersonal, and hyperpersonal interaction. Communication Research, 23(1):3–43, 1996.

[30] M. Fiedler. Algebraic connectivity of graphs. Czechoslovak Mathematical Journal, 23(2):298–305, 1973.

[31] M. Belkin and P. Niyogi. Laplacian eigenmaps for dimensionality reduction and data representation. Neural Computation, 15(6):1373–1396, 2003.

[32] J. Duprez, M. Stokkermans, L. Drijvers, and M. X. Cohen. Synchronization between keyboard typing and neural oscillations. Journal of Cognitive Neuroscience, 33(5):887–901, 2021.

[33] F. Richlan, J. Schubert, R. Mayer, F. Hutzler, and M. Kronbichler. Action video gaming and the brain: fMRI effects without behavioral effects in visual and verbal cognitive tasks. Brain and Behavior, 8(1):e00877, 2018.

[34] Counterpoint Research. Global Smartphone Average DRAM Hits Record 8.4GB in 2025, Memory Shortages Set to Hinder Growth, February 26, 2026. [Online]. Counterpoint Research.

[35] Omdia. Smartphone memory polarization deepens amid surging component costs, July 2026. [Online]. Omdia. The article reports a Q1 2026 global average of 8.3 GB DRAM and 279 GB NAND storage.

[36] Conversational data set used to create the evaluation word stream for IntentSpider and the comparison keyboards.
https://github.com/IntentSpider/datasets/blob/main/intentspider-initial.zip.

[37] Conversational data set used to train the Global Person Mode in the Research Preview.
https://github.com/IntentSpider/website/blob/main/assets/dataset-another.txt.

[38] IntentSpider Webnet. The Story of IntentSpider and Why Spider? July 30, 2026.
https://intentspider.github.io/website/about.html.

[39] All experiment resource files.
https://github.com/IntentSpider/datasets/tree/main/resources/resources.
[40] IntentSpider 500 word evaluation results.
https://github.com/IntentSpider/datasets/blob/main/resources/resources/500%20words/intentspider.xlsx.
[41] Samsung Keyboard 500 word evaluation results.
https://github.com/IntentSpider/datasets/blob/main/resources/resources/500%20words/samsung.xlsx.
[42] Google Gboard 500 word evaluation results.
https://github.com/IntentSpider/datasets/blob/main/resources/resources/500%20words/gboard.xlsx.
[43] Microsoft SwiftKey 500 word evaluation results.
https://github.com/IntentSpider/datasets/blob/main/resources/resources/500%20words/swiftkey.xlsx.
[44] OpenBoard 500 word evaluation results.
https://github.com/IntentSpider/datasets/blob/main/resources/resources/500%20words/openboard.xlsx.
[45] Other output and miscellaneous files used for the evaluation and validation section.
https://github.com/IntentSpider/datasets/tree/main/resources/resources/other%20files.
[46] IntentSpider web portal.
https://intentspider.github.io/website/.
[47] IntentSpider Interactive Research Preview.
https://intentspider.github.io/website/playground.
[48] IntentSpider C++ engine source code.
https://github.com/IntentSpider/IntentSpiderEngine.